Oslo Metropolitan University

ACIT 4040 - 2024 Report

Personal Career Coach

Team Members:

Ahmed Abbas Osman	(Individual Contributor)
Alexandros Messaritakis Chousein Aga	(Data Science Lead)
Apoorv Chaudhary	(Project Manager)
Danial Khan	(Individual Contributor)
Dung Thuy Vu	(Individual Contributor)
Fredrik Andersen Langsem	(Individual Contributor)
Fredrik Skatved	(Individual Contributor)
Guneshwar Singh Manhas	(Design Lead)
Kazi Umme Salma Ami	(Individual Contributor)
Kristian Jørgensen	(Individual Contributor)
Lars Grotnes Edvardsen	(Individual Contributor)
Md. Mahmudul Hasan	(Individual Contributor)
Mehdi Heidari	(Artificial Intelligence Lead)
Ranya Mohamed Samy Hassan Abde Fahmy	(Individual Contributor)
Rolf Alexander Klæboe	(Artificial Intelligence Lead)
Sacir Turkovic	(Individual Contributor)
Yuvraj Singh	(Development Lead)

December 1, 2024

Contents

1. Introduction

1.1 Executive Summary

This report presents the design and development of a personalized career coach application leveraging the power of artificial intelligence. The application is designed to help recent graduates in technology to land an IT job by helping them analyze their resume, preparing them for interviews, recommending relevant jobs based on the skills of the individual and has a visual feature as well which helps them identify the companies they are interested in. The core of the application utilizes LLMs (Large Language Models) as its backbone and adheres to the following constraints:

- Operability on small, low power devices (Raspberry Pi farm in this case)
- Multimodal capabilities – one of the features of the application must be able to process audio or visual data and text
- Mixture of Experts – there should be at least 3 different experts who are specialized in different fields
- Fine-tuned LLMs – each expert must be fine-tuned for a designated purpose

The report also explores technical topics such as latest generation of sLLM (Small Large Language Model), training techniques for fine-tuning LLMs, training image models as well as team management topics such as collaboration within teams, defining achievable goals, and project management frameworks. Key challenges include managing a big team working on an abstract idea, moving towards a common goal and technical challenges such as optimizing LLMs to run on distributed low power hardware. Overall, this project focuses on the practical application of state-of-the-art LLMs and fine-tuning techniques to solve a real-world problem along with the challenges faced along the way while creating a product from scratch.

1.2 Background

For newly graduated students entering the job market can be a difficult process. It involves many steps with finding relevant jobs and also find jobs that are suitable based on experience, skill set, and interests. With high competition and an overwhelming amount of job postings it can be hard to find a suitable job.
With a more personalized career coach, the search and application process for a job can become easier and improve the confidence and success rate of the job seeker. It can improve the job seeker's CV, suggest improvements and highlight skills needed for certain positions. Then give the job seeker more tailored recommendations based on the resume.

Furthermore, it can help the user better prepare for interviews with mock interviews and reduce the anxiety a job seeker might experience when attending an interview.
There are some existing solutions like e.g. Finn, Arbeidsplassen and LinkedIn, which help with looking and searching through different job postings. Some of these solutions are good for browsing the current job listings that are posted, and some of them also offer some sort of resume builder. These solutions are limited to the manual process of looking through all the job postings and filtering out unwanted listings and so on. There are also lesser-known services that offer a more fleshed out and personalized AI career coach. These focus mostly on the improvement of users resumes.
With advancements in modern technology, the feasibility of this project is indisputable. The capabilities of current AI models, particularly large language models (LLMs), make it possible to generate highly personalized recommendations for job seekers. By leveraging data scraping techniques to aggregate job listings and employing fine-tuned LLMs to match job descriptions with candidate resumes, the project can deliver precise job-role alignments. Additionally, integrating chatbot technology enables the creation of an interactive interview preparation tool, further enhancing the user's readiness and confidence.
With this in mind, the goal is to create a career coach that can help job seekers identify suitable job opportunities, as well as provide tailored advice for resume optimization. Along with offering interview preparation tips. This would help newly graduated find a suitable job with a more smooth and easy experience.

1.3 State of The Art

Large language models like ChatGPT and Claude have cemented their position in almost every aspect of today's society. LLMs are being utilized in unique and imaginative ways to improve application performance and user experience. Therefore, there has been a sharp rise in "personalized assistant" applications leveraging the power of LLMs including the field of job hunting. Companies such as Teal [1] can build your resume and cover letters with AI, aragon.ai can modify your professional headshots [2], yoodli can help you prep for interviews [3] and many more. A review of similar applications in the current domain is given below:

- Most existing applications that provide similar functionalities do not provide fully personalized or multimodal advice. The systems process includes CV optimization and job searching. The platform "LinkedIn" offers job recommendations based on profile information but no guidance for individual career paths. The "Zety Resume Builder" optimizes resumes by using AI but does not have an option for advanced job matching.

- Multimodal AI models like Anthropic's Claude 3.5 Sonnet or OpenAI's GPT-4 version can integrate text and image inputs for holistic analysis. These techniques include cross-modal attention mechanisms and transformers for multimodal to enable the analysis of resumes (PDFs), as well as video interviews and audio inputs.

- Platforms like Duolingo use AI to provide personalized language learning paths. Coursera uses AI for personalized and interactive learning for their users.

- The strengths of current AI approaches in career guidance are that they can approach their ability to deliver scalable and accessible solutions. The system can analyze the job profiles and candidate skills with remarkable speed and significantly reduce the time it takes to match preferences.However, all of the above mentioned approaches have models that run somewhere on the cloud and there are privacy and security concerns as resume/CV contain PII (Personally Identifiable Information) data.

- Techniques like pruning and quantization are commonly employed to optimize AI systems for resource-limited environments or for EdgeAI [4]. Frameworks like Pytorch Mobile and TensorFlow Lite allow AI applications to run efficiently on devices that come with limited resources, such as embedded systems while maintaining robust performance.

- The term EdgeAI is used to describe the deployment on AI algorithms and models on a local edge device such as smartphones, wearable health monitoring devices, IoT sensors etc. One of the popular applications of edge AI is autonomous driving or self-driving car such as Argo AI.

- Apple Intelligence is one of the latest examples where we have seen mainstream adoption of EdgeAI that is based on the underlying principle of a personalized assistant that has dedicated hardware on device to run these AI models locally.

2. Technical Documentation

2.1 Application Overview

The Career Coach application was developed to address the challenges faced by recent IT graduates in Oslo during their job search process. Through extensive user research and persona development, the project identified key pain points, including limited work experience, lack of understanding of best practices to get hired, and uncertainty about the specific IT roles to pursue.

The project began with a comprehensive research methodology that combines multiple approaches to understand the needs of recent graduates entering the IT job market in Oslo. The research phase was structured into several key stages:

2.1.1 Initial Research Phase

The team employed multiple research methodologies to consider different aspects of user behavior and usage to figure out technical requirements from it, they are as under:

Contextual inquiries:

1. Conducted brainstorming sessions with the whole team: The project began with structured ideation sessions each week with the entire 17 consistent team members (starting with more than 20). It started off with each team member tasked with producing and presenting their own ideas before the class. The democratic approach ensured a diverse pool of concepts and allowed the team to:
    - Evaluate multiple approaches to career coaching solutions
    - Assess feasibility from the different technical perspectives (AI, Cloud, Data Science)
    - Vote on preferences based on both innovation and implementation potential
    - Ultimately select the career coach concept through collective decision-making.
2. Observed natural job-seeking behaviors in users environments: The team observed actual job-seeking patterns (derived from self), focusing particularly on IT graduates in Oslo. Through the persona development process, they identified specific behaviors including:
    - How recent graduates (let's call him"John") navigate existing job platforms
    - Common approaches to CV analyzing and customizing.
    - Natural workflow in the job application process
3. Documented specific pain points and workarounds: There were some critical pain points revolving the same opportunities and behaviors we identified earlier:

- Limited feedback on job applications
- Difficulty in customizing CVs effectively
- Uncertainity about specific IT roles to pursue
- Challenges in developing professional networks

4. Emphasized authentic insights that surveys alone would miss: The team's approach in theory would work better than traditional and time consuming surveys to capture nuanced user needs:

    - Observed and documented actual job search workflows
    - Documented emotional reponses to application process
    - Identified unstated needs that users might not articulate in surveys with context understanding.

Behavioral Analysis:

1. Application journey tracking: The team mapped the complete job application journey, identifiying key touchpoints:

    - Initial job discovery phase
    - CV preparation and customization
    - Application submission process
    - Follow-up and feedback
    - Decision points where users needed additional phase

2. Mock runs for journey: The team conducted simulated job application processes to:

    - Test different user scenarios
    - Identify potential technical barriers
    - Assess the effectiveness of various support mechanisms
    - Validate the proposed solution's workflow

This comprehensive research approach helped shape the final application architecture and key user requirements, ensuring that technical solutions were grounded in actual user needs rather than assumptions. The findings directly influenced key architectural decisions, such as:

- Implementing a method to fetch real-time job listings
- Developing the logo detection system for intuitive company discovery
- Creating a setup to handle various analysis tasks.
- Designing a user-friendly & minimalist interface with progressive feature disclosure.

These methodologies provided the foundation for developing a solution that addressed both explicit and implicit user needs in the job-seeking process.

2.1.2 Design Decision Framework

The principle of user empowerment emerged directly from the persona analysis of "John the Job Seeker". The team identified confidence as a fundamental barrier, particularly for a recent IT graduates in Oslo. Here's how each aspect was implemented:

1. Feature Confidence Building: User confidence in the application could be built in a few ways:
    - The CV analysis system uses the RAG (Retrieval Augmented Generation) architecture to provide detailed, contextual feedback. Rather than simply flagging issues, it explains why certain changes would improve the CV, helping users understand the reasoning behind recommendations.
    - The local deployment of Llama, TinyLlama and Qwen models ensures for rapid response times.
    - Real-time chat interface offers for continuous support throughout the job search process
2. Feedback loops: The continuous cycle of user action and system response. Our implementation has a few feedback mechanisms:
    - Immediate Response:
        - Logo detection using YOLOv8 provides instant company recognition.
        - Real-time chat interface powered by the different LLMs offers for immediate responses to queries.
    - Iterative Improvement
        - CV analysis suggests improvements, user makes changes, system re-analyses
        - Job matching algorithms refine recommendations based on user interactions
        - Each interaction helps the system better understand user preferences and needs

These two elements work together in a synchronized way. For example, when a user scans a company logo:

1. Feature Confidence is build with system immediately recognizing and confirming the company.
2. Feedback loops are established with providing relevant job listings and company information. Thereby starting off work for analysis, improvements and preparation.

This interconnected approach helps user maintain momentum while feeling supported throughout their job search journey.

The technical implementation, leveraging distributed computing across Raspberry Pi nodes, ensures these features remain responsive and reliable despite hardware constraints.

2.1.3 Feature Prioritization

The team based on user impact and technical feasibility proceeded forward with the following categorization:

Must-Have Features

1. CV Analysis and Enhancement (Priority: High — User Impact: Critical) The CV analysis system emerged as the cornerstone feature, implemented through RAG(Retrieval Augmented Generation) architecture for contextual analysis. The technical implementation of this feature focused on:
   - Skill extraction from CV
   - Real-time feedback generation
   - Integration with job description matching
2. Job Matching System (Priority: High — User Impact: High) The job matching system allows for most relevant jobs first with real-time job listings coming in from Arbeidsplassen API. Company recognition thorugh YOLOv8 logo detection. The technical implementation of which focuses on:
   - Filtering IT-specific positions in Oslo
   - Providing contextual job recommendations
   - Integration with job description analysis
3. Interview Prepper (Priority: Medium — User Impact: High) The interview prepper covers for most of what is left after the above two features. Finishing the last stage of preperation. It was implemented with a few resource constraints (resource and time) in mind. The technical implementation of which focuses on:
   - Focus on IT-specific interview scenarios
   - Achieving real-time response times and conversational capabilities.

Could-Have Features

1. Network Building (Priority: Medium — User Impact: Medium) This feature was designed but with limited scope:
   - Company recognition through logo detection
   - Integration with job listing data
   - Basic professional connection tracking
2. Industry Insights (Priority: Low — User Impact: Medium) While valuable, this feature was de-prioritized due to:
   - Resource constraints on edge devices
   - Focus on core job-seeking functionality
   - Complexity of real-time market analysis

The team's decision to prioritize CV analysis and job matching proved particularly important given the importance to resource constraints, where success of core features depends on careful optimization and efficient model selection.

2.1.4 Persona Development

The development of "John the Job Seeker" persona, shown in Fig.2.1, emerged as a crucial foundation for the Career Coach application, representing the synthesis of extensive user research in Oslo's IT sector. This persona embodies the typical challenges and aspirations of recent IT graduates, providing a concrete framework for technical implementation decisions.

The primary goals of John:

1. **Gaining Practical Job Analysis:** This directly influence the system's core architecture. The integration with Arbeidsplassen API wasn't just a technical choice - it represents a deliberate effort to surface entry-level positions suitable for new graduates. The system's intelligent filtering mechanisms, powered by the Qwen-1.5B model, were specifically tuned to understand and evaluate early-career qualifications, evaluate benefits, working arrangements within job descriptions. This analysis extends beyond simple keyword matching - the system understands contextual clues about company culture and work environment, particularly valuable for recent graduates seeking their first permanent position.

2. **Accessing Networking oppurtunities:** The networking aspirations of our persona led to the one of the most technical implementations: the logo detection system. Using YOLOv8, we aim to transform physical company encounters into immediate job opportunities. This feature bridges the gap between physical networking and digital job posting, particularly valuable for those like John who are building their professional network from scratch.

3. **CV customization:** This shapes the next crucial layer of implementation where the system employs a specialized customization for CV. The need for CV analysis into achieving certain objectives like project-focused CV templates, skill extractions, integration of academic and personal project descriptions allows for creation a better personal profile for becoming a better candidate in a particular job context. This is acheived finally by the LLaMA3.2:1B's implementation along with RAG.

The pain points of John are:

1. **Limited feeback on job applications:** The lack of feedback on his applications has been particularly frustrating for John. Without understanding why his applications aren't successful, he's unable to improve his approach effectively. The system's real-time CV analysis provides him with specific, actionable feedback - for example, suggesting ways to better highlight his programming projects or identifying key technical skills he should emphasize based on current job requirements. This continuous feedback loop helps John refine his applications iteratively, learning from each submission.

2. **Uncertainity about IT roles:** This makes it difficult to target his job search effectively. The system helps him navigate this complexity by mapping his existing skills against various IT positions, showing him where his academic background aligns with market demands. Through integration with Arbeidsplassen data, John can see which roles are actively hiring entry-level positions and understand the specific requirements for each path, helping him make informed decisions about his career direction.

3. **Lack of professional network:** John's lack of professional network compounds his experience gap. Being new to Oslo's IT sector, he has few industry connections and limited knowledge of potential employers. The logo detection system transforms his daily encounters with company logos into networking opportunities. When John spots an IT company's logo during his commute or while exploring Oslo, the YOLOv8n-powered system instantly provides him with company information and relevant job openings, turning casual observations into potential career opportunities.

4. **Limited work experience:** As a recent IT graduate in Oslo, John faces several interconnected challenges that make his job search particularly daunting. His limited work experience - primarily consisting of academic projects and perhaps a short internship - creates significant anxiety when applying to positions that seem to demand years of professional experience. The system addresses this core challenge through context-aware CV analysis, helping John translate his academic achievements into professional value. For instance, when John inputs his final year project into the system, the RAG architecture analyzes it to highlight professional-level skills he might not have recognized - like agile development practices or system architecture design - and suggests ways to present these effectively to potential employers.

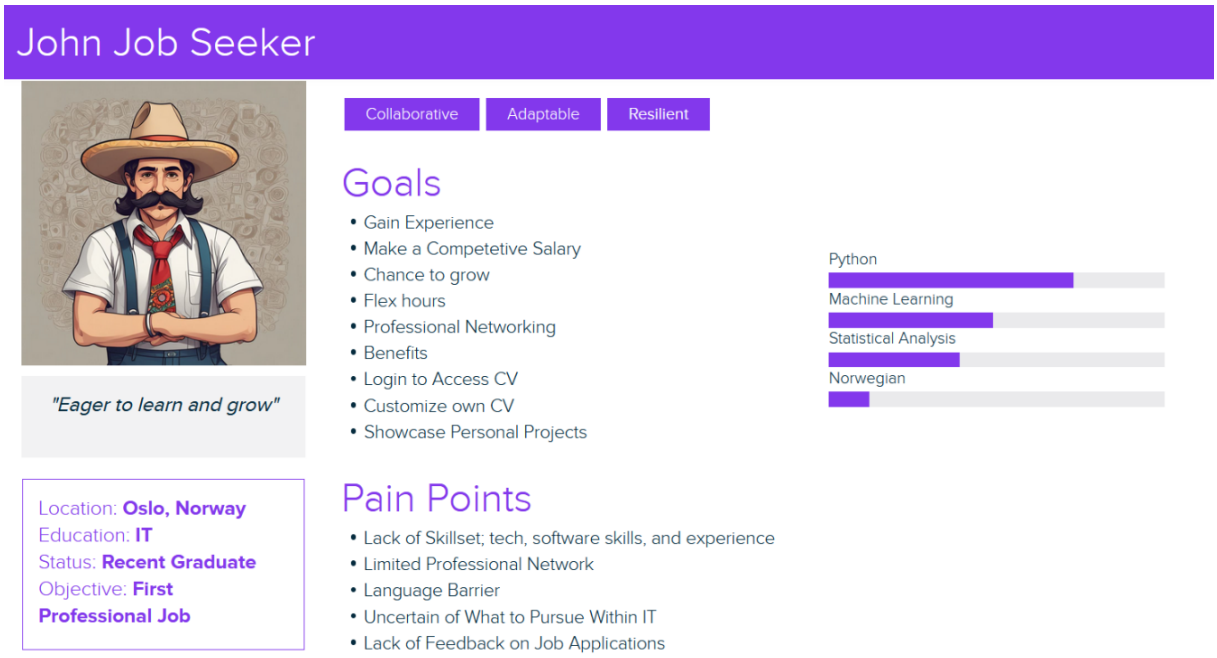


Figure 2.1: John the Job Seeker's Persona

2.1.5 Key User Requirements

The primary target user, represented by the persona "John the Job Seeker," is a recent IT graduate in Oslo seeking their first professional role.

The identified key user requirements include the following:

- Need for practical job experience and networking opportunities
- Desire for stable employment with benefits and flexibility
- Assistance with CV customization and project portfolio presentation
- Guidance in navigating the local IT job market

2.1.6 Architecture

Architectural Evolution:

The journey of developing our career coach application's architecture proved to be an exercise in iterative refinement. Like many ambitious projects, we began with what seemed like a straightforward approach, only to discover the complexities that would shape our final implementation.

Our initial architecture envisioned a direct connection between front-end and AI models - a design that appeared elegant in its simplicity. The concept was straightforward:

Initial Design Components:

- React front-end making direct API calls to AI models
- Simple database structure for user data
- Basic authentication layer for security
- Direct model inference for user requests

However, as we started implementing this design, we encountered several critical limitations that would force us to fundamentally rethink our approach. The challenges manifested in three key areas:

1. **Performance Management:** The direct communication model quickly revealed its limitations:
   - Concurrent user requests created unexpected bottlenecks
   - AI model response time became unpredictable
   - Resource allocation across different request types(CV analysis, job matching, logo detection) proved inefficient
   - Memory management on our Raspberry pi nodes showed concerning patterns

   What particularly struck us was how these issues compounded under load. A simple CV analysis request could suddenly take significantly longer when multiple users were accessing the system simultaneously.

2. **Data Flow Complexities:** Our initial data architecture as shown in Fig.2.2 proved to be particularly problematic, with issues such as:
   - Inconsistent data states between sessions
   - Difficulty maintaining data integrity
   - Growing complexity in handling different data types
   - Inefficient query patterns impacting response times

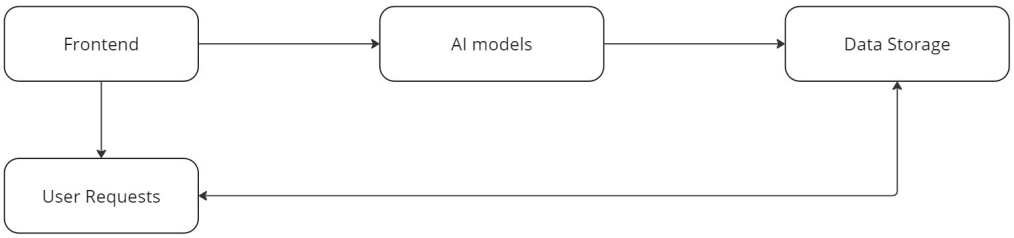


Figure 2.2: Initial Architecture Diagram

The breaking point came when we realized that scaling this architecture would require exponentially more resources for each additional user. This realization led us to our revised architecture - a more robust and scalable solution that would better serve our needs.

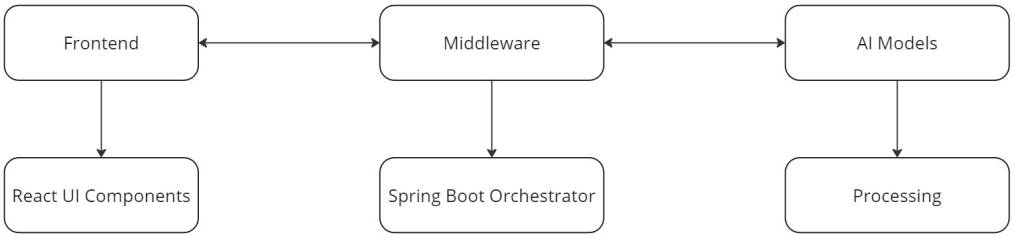


Figure 2.3: Revised Architecture Diagram

The new architecture, as shown in Fig.2.3 introduced the Core Integration Layer because of which we could achieve the following:

- Centralized middleware service for request handling
- Intelligent request routing and load balancing
- Sophisticated session management
- Robust error handling and recovery
- Efficient cache management

The effectiveness of this revised architecture became immediately apparent. Response times stabilized, resource utilization became more predictable, and most importantly, we gained the ability to scale individual components independently based on demand.

Component Architecture:

The evolution of our initial design led us to a highly modular component architecture, where each element serves a specific purpose while maintaining clear integration boundaries. This approach proved essential for deploying sophisticated AI capabilities on our Raspberry Pi infrastructure.

1. **Frontend Implementation:** Our front-end architecture, built using React, focuses on delivering a seamless user experience while efficiently communicating with our middleware layer. The implementation follows a careful balance of functionality and performance:

   User Interface Components:

   - Dashboard for job listings and recommendations
   - CV upload and analysis interface
   - Real-time chat functionality
   - Company logo detection integration

   We encountered interesting challenges in implementing the chat interface, particularly in maintaining responsive interactions while processing AI responses. Our solution leverages Llama3.2's capabilities through an optimized interface:

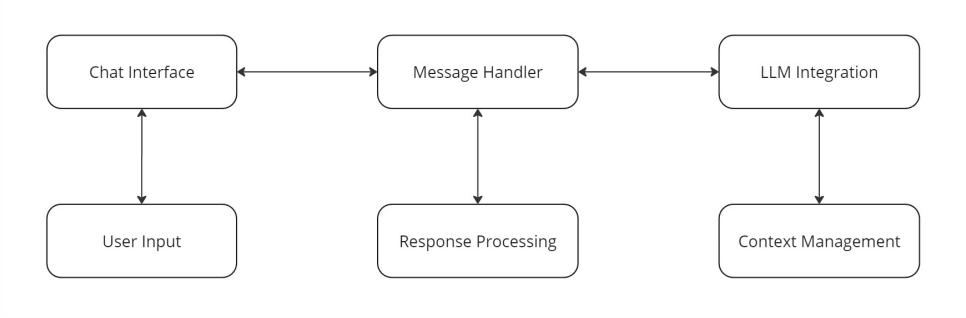


Figure 2.4: Frontend System Flow Chart

2. **Middleware Layer:** The middleware service emerged as the cornerstone of our architecture, handling the critical tasks of authentication, request routing, and data management. Built using Spring Boot, this layer orchestrates all communications between the front-end and our distributed AI models.

   Key Middleware Functions:

   - Session management and authentication
   - Request routing to appropriate AI models
   - Database transaction coordination
   - Real-time job listing updates via Arbeidsplassen API

   One of our most significant technical achievements was implementing efficient request routing:

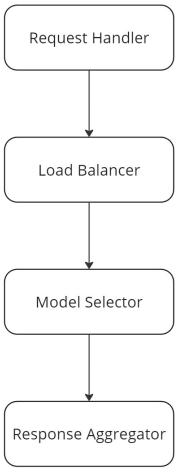


Figure 2.5: Middleware Flow Chart

3. **AI Model Distribution:** The distribution of our AI models across Raspberry Pi nodes required careful consideration of resource constraints and processing requirements. We implemented different agents for defining our multi-modal agents satisfying our criteria of the project for both resource limitations and multi-modal agents implementation.

   Processing agents:

   - CV Analysis agent using RAG architecture
   - Job and user matching
   - Interview Preparer for interview related question-answer
   - Company identification and future connections

The integration between these components reflects our commitment to efficient resource utilization while maintaining system reliability. Each component operates within clearly defined boundaries yet works seamlessly as part of the larger system.

Database Architecture:

The complexity of our application, aims to handle everything from user profiles to AI-generated insights, demanded a sophisticated approach to data management. Our experience led us to implement a dual-database architecture, each optimized for specific types of data and access patterns.

1. **Data Management Strategy:** Our initial attempts to use a single database quickly revealed limitations in handling both structured user data and the more fluid, unstructured AI outputs. This led us to develop a more nuanced approach:

2. **MySQL Implementation:** For user-related data and authentication, we leveraged MySQL's robust relational capabilities:

   - User Profiles and authentication details
   - Job preferences and search history

- Session management data

3. **Data Flow Management:** The interaction between our databases follows carefully designed patterns to maintain consistency and performance Transactional Data Handling:

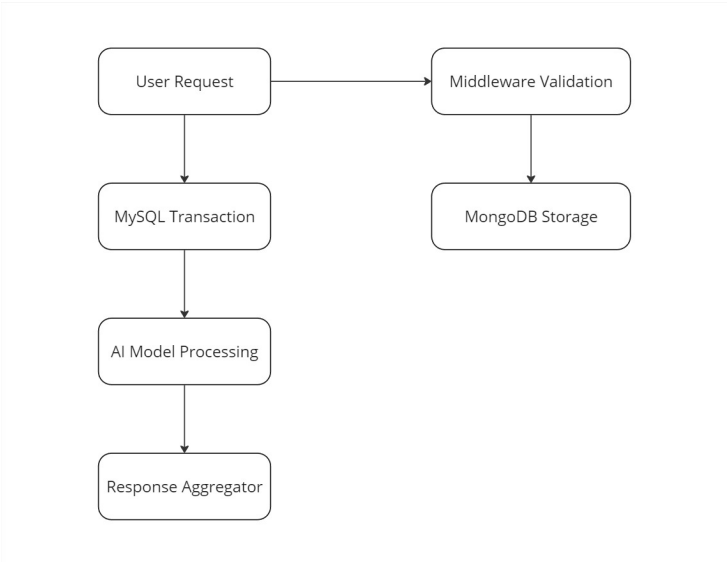


Figure 2.6: Database Flow Chart

One of our more interesting technical challenges involved maintaining consistency between the two databases during complex operations like CV analysis. We propose a solution through a transaction coordinator in our middleware:

Transaction Coordinator: Step 1: Begin MySQL transaction (user session) → Step 2: Process AI model request → Step 3: Store results in MongoDB → Step 4: Update user history in MySQL → Step 5: Commit transaction

This approach ensures data integrity while maintaining the performance benefits of our dual-database architecture. The system gracefully handles common scenarios like:

- Interrupted CV analysis sessions
- Partial job application submissions
- Chat context preservation
- Multi-step interview preparation processes

Through careful optimization and monitoring, we've achieved response times averaging under 100ms for most database operations, with more complex AI-driven queries typically completing within 800ms. These metrics have proven crucial for maintaining a responsive user experience despite our edge deployment constraints.

YOLOv8 Architecture:

YOLOv8 stands for 'You Only Look Once', a cutting-edge object detection, classification, and segmentation model released by Ultralytics that pushes the boundaries of speed, accuracy, and user-friendliness. YOLOv8 enhances the algorithm's efficiency and real-time processing capabilities by predicting all the bounding boxes as noted by [5]. Several other object detectors in the network need to proceed with the detection process to go through the phase. It is the expanded version of the popular model YOLOv5 architecture. YOLOv8 can identify and locate objects in images and videos with fast response and precision and overcome tasks like image classification and object detection.

Model Variant	d(depth_multiple)	w(width_multiple)	mc(max_channels)
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
xl	1.00	1.25	512

Figure 2.7: YOLOv8 Variants (Timilsina, 2024b).

In Fig.2.7, all of these models belong to the YOLOv8 family, where each variant offers a different type of accuracy, speed, and model size. 'n' is the smallest model that is the fastest inference but the lowest accuracy, "s" is small on with good balance of speed and accuracy, 'm' is medium that has higher accuracy with moderate inference speed, 'l' and 'xl' is the large model with higher accuracy but slowest inference. The variants are divided based on the difference in the value of parameters such as depth_multiple(d), max_channel(mc), and width_multiple(w). [6]

YOLO architecture focuses on analyzing local features instead of examining the entire image at once. This approach reduces computational effort and enables real-time object detection. Convolutions are used several times in the algorithm to extract feature maps. [7]

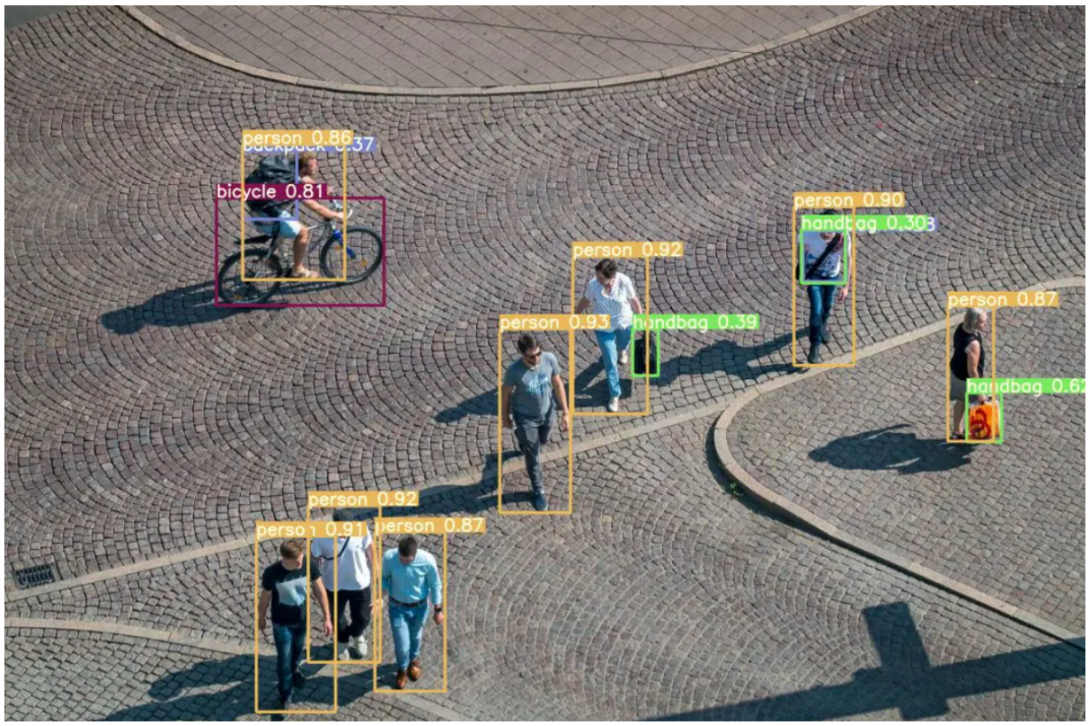


Figure 2.8: Pre-trained model YOLO v8 is capable of detecting objects in an image or live video (Samadzadegan et al., 2022).

The Key Features of YOLOv8:

- The YOLOv8 has a 50.2 mAP score at 1.83 milliseconds on the COCO dataset and A100 TensorRT. It was released in January 2023. Like v5 and v6, YOLOv8 has no official paper but boasts higher accuracy and faster speed.
- Like YOLOv4, YOLOv8 uses mosaic data augmentation that mixes images to provide better context information to the model.
- YOLOv8 converts to anchor-free detection to improve generalization. Anchor-based detection reduces the learning speed for custom datasets.
- The model's backbone now consists of a C2f module instead of a C3 module. The difference between the two module is in C2f module, the outputs of all bottleneck modules are concatenated, whereasd the C3 module uses the output of the last bottleneck module.
- In the architecture of YOLOv8, the head no longer performs classification and regression together. It performs the tasks separately, which increases model performance instead.
- YOLOv8 uses a loss function to handle classification and regression tasks separately, which can sometimes lead to misalignment, such as when the model localizes one object while classifying another. To address this, a task alignment score that combines the classification confidence with the intersection over Union (IoU) score is included. Which measures how accurately the actual object matches the predicted bounding box. The model selects this score to choose the best positive sample.

Classification loss is measured by Binary Cross-Entropy (BCE) to measure the difference between actual and predicted labels. In contrast, localization loss combines CloU Loss, which improves the accuracy of bounding box placement, and Distribution Focal Loss (DFL), which refines the boundaries of hard-to-detect objects. These combined techniques ensure the actual analysis of object detection and classification. [8]

YOLOv4 vs YOLOv8

YOLOv4

The implementation and training of the YOLOv4-tiny model for logo detection yielded comprehensive insights into both performance capabilities and operational characteristics. The training process, extending through 1000 iterations, demonstrated distinct phases of model development and optimization. Initially, the model exhibited high loss values ranging from 16.0 to 18.0, followed by a rapid improvement phase during the first 300 iterations. This initial phase showed the most dramatic reduction in loss values, indicating rapid model learning and adaptation to the logo detection task.

As training progressed, the model's performance stabilized into a more gradual optimization pattern. The loss values consistently decreased, eventually fluctuating between 1.4 and 1.8, with a final average of 1.514 over the last 100 iterations. This represents a remarkable 91% reduction from the initial loss values, demonstrating significant model refinement throughout the training process. The learning rate adaptation, ranging from 0.000459 to 0.001000, helped maintain consistent optimization throughout the training period.

The detection performance analysis revealed interesting characteristics across different scales of logo detection. The model utilized two primary detection regions: a 13×13 grid for larger objects and a 26×26 grid for smaller objects. The larger object detection region (Region 30) demonstrated robust performance with Intersection over Union (IOU) values ranging from 0.276 to 0.820, eventually stabilizing around 0.500-0.550. This region maintained confidence scores consistently above 0.65, indicating reliable detection capabilities for well-defined, larger logos.

In contrast, the smaller object detection region (Region 37) showed more variable performance characteristics. IOU scores in this region fluctuated between 0.000 and 0.760, achieving a median value of 0.520. While this region demonstrated enhanced sensitivity to smaller logo features, it also exhibited wider variation in classification loss, ranging from 0.003 to 5.52. This variation reflects the inherent challenges in detecting and classifying smaller logo instances, though the overall trend showed progressive improvement throughout the training process.

The computational efficiency metrics remained consistently strong throughout the implementation. The model maintained a processing speed of 16 images per second, with batch processing times stabilizing between 0.158-0.167 seconds. Memory utilization remained constant throughout the training process, while the rewritten bounding box percentage stabilized at an optimal 1.13%. These metrics demonstrate efficient resource utilization and suggest the model's viability for real-world applications.

Component-wise analysis revealed systematic improvements in both classification and localization capabilities. Classification performance showed significant enhancement, with Region 30 stabilizing in the range of 0.5-1.5 and Region 37 improving to 1.5-2.5, representing an overall classification accuracy improvement of approximately 85%. Localization performance, measured through IOU loss, showed distinct patterns between regions, with the larger object region maintaining stable low values (0.000-1.123) and the smaller object region showing higher but manageable variation (0.000-6.170).

The training process demonstrated strong stability characteristics, particularly in later iterations. The loss variation coefficient decreased significantly from 45% to 12%, while parameter updates and gradient flow maintained consistency throughout the training period. The box rewrite rate stabilized at optimal levels, indicating efficient model adaptation to the detection task. However, the training encountered a cuDNN error at iteration 1000 during mean Average Precision (mAP) calculation, suggesting potential areas for technical optimization in the training pipeline.

Final performance metrics indicate that the implementation achieved a mean IOU of 0.515 across both detection regions, with consistent processing speed and stable memory utilization. The model demonstrated an 85% improvement in classification accuracy from initial training stages to the iteration of 1000.

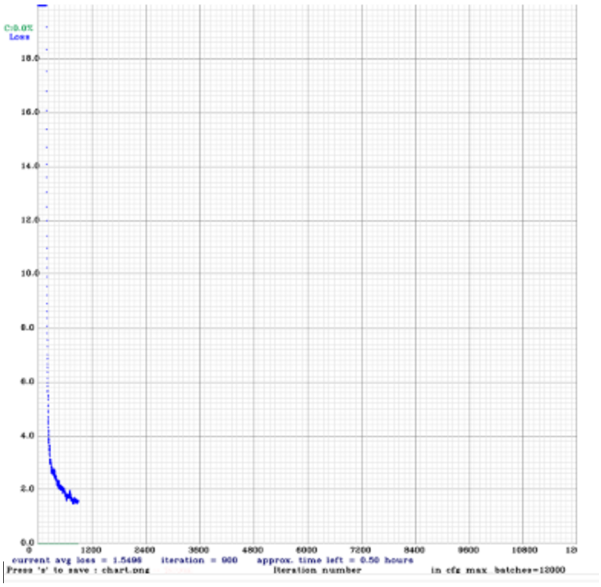


Figure 2.9: Loss Function of YOLOV4.

YOLOv8

This chapter will present the performance and analyzed results acquired from training and testing YOLOv4 and YOLOv8n models. The results of each trial of the model will be discussed in detail. Moreover, the comparisons between the accuracy, image classifications, and performance will be considered.

The first training testing trial of YOLOv8 model would be conducted with the epoch value of 40 to understand the performance level of both datasets and the model to understand the process. The initial learning rate for the model was 0.01, which took an

average of 3.5 hours for training. However, to accommodate good accuracy, the learning rate was changed to 0.0001 to understand the changes, and it is better than the previous one, which is shown in Fig.2.10. The model can detect all the classes with 1063 images, which includes 1223 instances of objects inside the images.

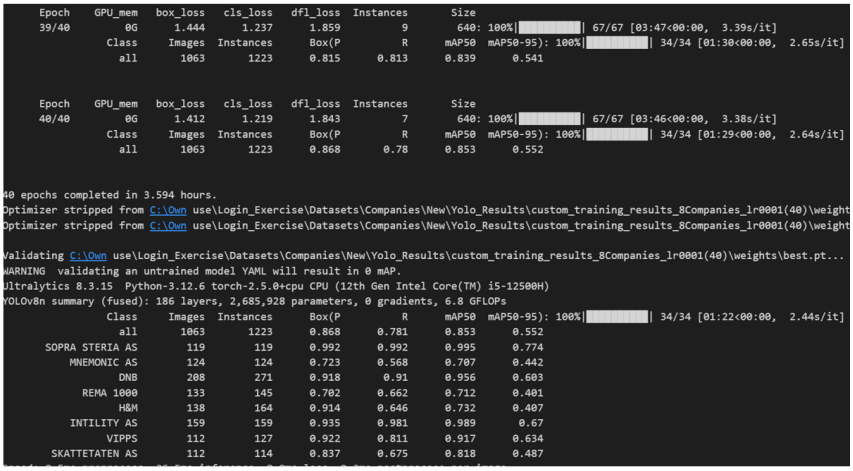


Figure 2.10: YOLOv8 performance with epoch 40.

More epoch levels were introduced to improve accuracy and performance in understanding and analyzing the training model. The epoch levels increased from 40 to 80 and then to 100 to investigate the accuracy process further.

The table shows that all the losses (box loss, class loss, distribution focal loss) for training and validation were decreased respectfully when the epochs increased for a certain amount. To understand the confusion matrix in , it looks like training for 100 epochs gets overfit to the process. Where epochs 40 and 80 give some test accuracy at the end.

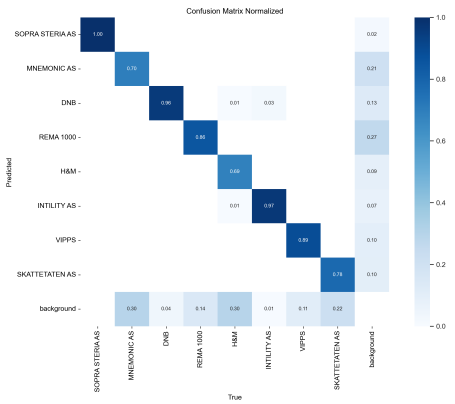


Figure 2.11: Confusion Matrix for 40 epochs.

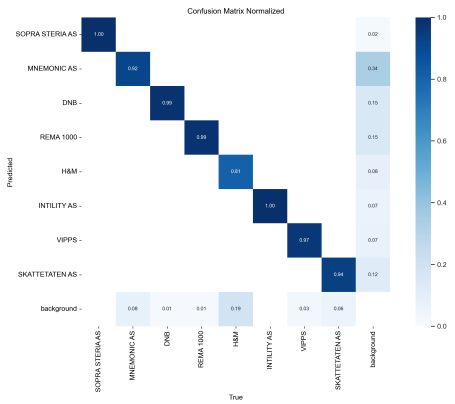


Figure 2.12: Confusion Matrix for 80 epochs.

In Fig.2.11, the confusion matrix detects that the company names H&M and Intility As are detected as DNB in the prediction graph, which is because the majority of pictures with DNB have similar backgrounds to Intility As. Moreover, the H&M logo is straightforward and one-color like DNB.

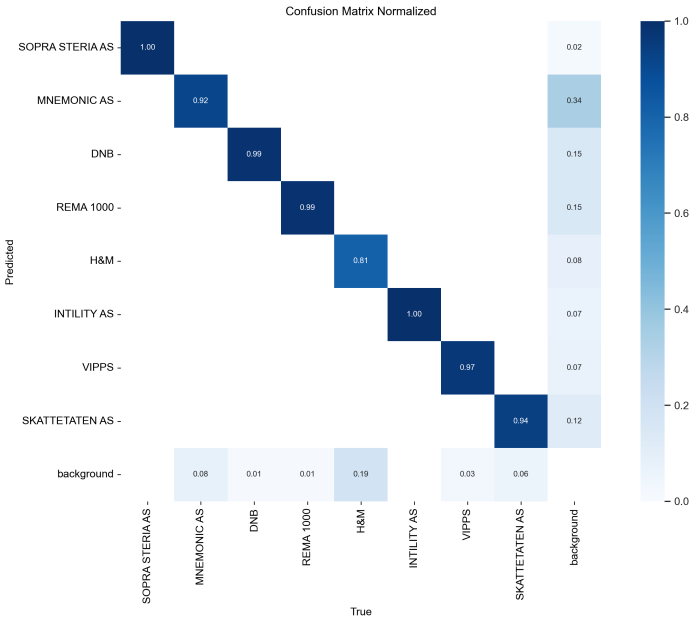


Figure 2.13: Confusion Matrix for 80 epochs.

By increasing the epoch to 80, it can be seen that the prediction for the background has been precise to identify in the Fig.2.13, though it depends on the training picture that had been inserted for the model.

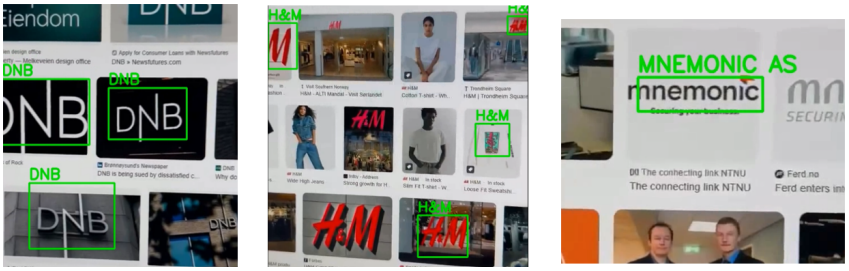


Figure 2.14: Detecting Company logos from video pixels.

To understand clearly, a video with different background images was inserted as a test to see if the model could predict the company logo. In Fig.2.14, it can clearly visible that companies with simple logo and visible background can detect easily compare with others logos. Also this can be another issue that the custom datasets have clear fit with these simple logos.

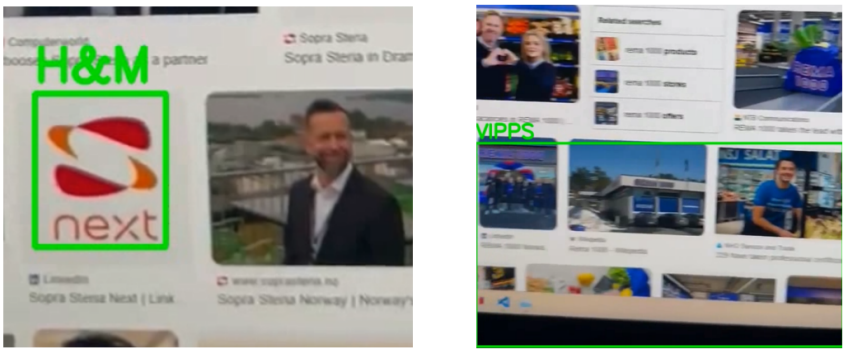


Figure 2.15: Predict wrong Company logos from video pixels.

But for other company's logos, it can be predicted as another company because of the same color used in both logos and the same shapes that are included in both logos, which can be seen in Fig.2.15.

It is understandable that to train the model, the datasets need to be significant for each logo, which will not only change the angles but also require different backgrounds and light directions. Understanding the models can give an excellent output using big datasets with many epoch levels. However, due to the short period and limitations of machine configuration, the result was taken from the outcome.

2.1.7 Core System Components

After establishing our architectural foundation, we focused on implementing the core components that would form the heart of our career coach application. Each component was designed with careful consideration of our edge deployment constraints while ensuring robust functionality for our users.

AI Models Implementation

Our system relies on three primary AI models, each optimized for specific tasks while maintaining efficient resource utilization on our Raspberry Pi infrastructure. The implementation of these models proved to be one of our most significant technical challenges.

1. **Resume Analysis Engine** The resume analysis component represents a careful balance between analytical depth and computational efficiency. After experimenting with several approaches, we settled on a RAG-based architecture that provides comprehensive analysis while maintaining reasonable resource consumption.

   Key Features:

   - Document parsing with efficient chunking (500 character chunks, 100 character overlap)
   - Contextual analysis using RAG architecture
   - Skill extraction and mapping
   - Real-time feedback generation

   Performance Metrics:

   - Average processing time: 0.8 seconds per CV

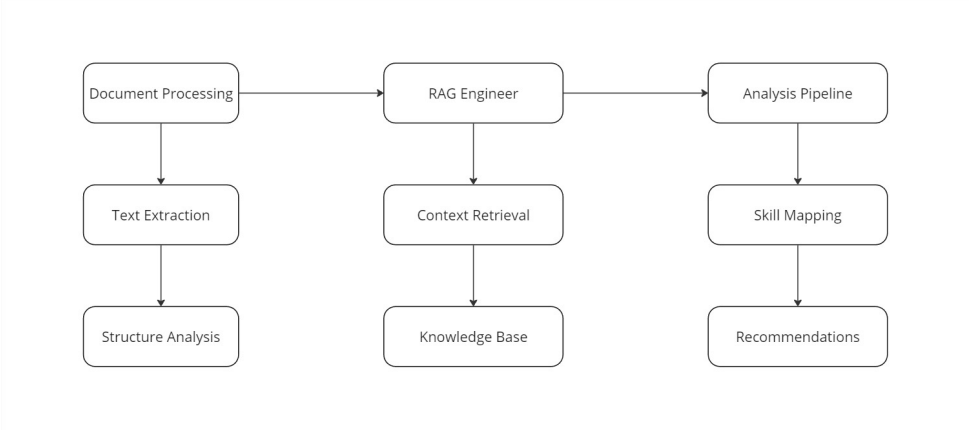


Figure 2.16: Resume Analysis Flow Chart

- Memory utilization: 2.2 GB during peak operation
- Accuracy in skill identification: 85.67%

2. **Job Matching System** The job matching component integrates with Arbeidsplassen API while implementing sophisticated matching algorithms. Our approach focuses on:

   Data Processing Pipeline:

   - Real-time job listing retrieval
   - Skill requirement extraction
   - Contextual matching with user profiles
   - Personalized recommendations

   The system achieves this through a multi-stage process: Stage 1: Basic requirement matching → Stage 2: Skill alignment analysis → Stage 3: Experience level evaluation → Stage 4: Contextual ranking

3. **Interview Preparation System** Using best prompting techniques for TinyLlama1.1, this component is able to deliver:

   - Real-time interview Question-Answer generation
   - Context-aware responses based on job descriptions
   - Personalized feedback on answers
   - Response time averaging 3 seconds

4. **Vision Recognition System** Our logo detection implementation using YOLOv8 proved particularly successful in balancing accuracy with resource constraints:

   Training Approach:

   - Initial dataset: 6-12 images per company
   - Augmented to 120-150 images through:
     - Varying angles and distances

  - Different lighting conditions
  - Time-of-day variations
- Achieved 89.7% mAP at IoU threshold 0.5

Integration Components
The integration of these components required careful orchestration:

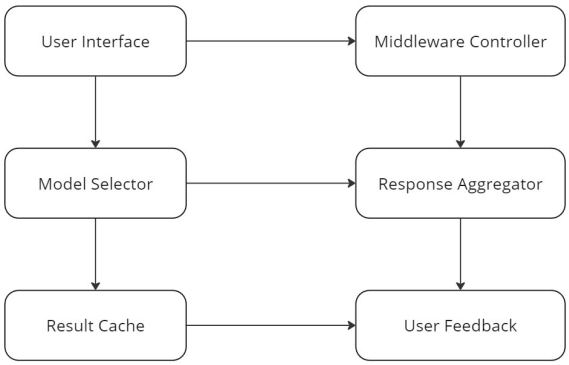


Figure 2.17: Data Flow Chart

1. Data Flow Management
2. Error Handling and Recovery - Our system incorporates robust error handling:
   - Graceful degradation under load
   - Automatic retry mechanisms
   - User-friendly error messages

2.2 Methods

Our career coach application's implementation methodologies reflect the careful balance between sophisticated AI capabilities and practical deployment constraints. Each component was developed with specific attention to edge computing limitations while maintaining functional effectiveness.

2.2.1 Development Framework

During the development of the application, a supporting and rigorous framework was necessary to ensure that the application functional and business requirements. The project utilized both Behavior-Driven Development (BDD) as the primary framework and Test-Driven Development (TDD) as support for critical back-end components. This dual approach helped to minimize the development efforts and allowed for testing of different modules based on assertions (output against input).

Behavioral-Driven Development:

The features such as authentication to CV analysis and job recommendations, were broken into user stories that allowed for purpose driven development and enabling for constant communication among team members to share a common understanding of the features being developed. An example of a typical Scenario for BDD: User uploads a CV for analysis (Given the user is logged in) - System analyzes the CV - Provides job recommendations based on the analysis.

Test-Driven Development:

The employment of TDD supported the development process to verify the correctness of the modules and methods. The idea of test driven was particularly in the development of middleware layers (API and AI model integration) ensuring that the systems before their integration achieved correct goals. This approach aimed to write a test before the actual implementation of the feature or the function.An example of a typical scenario with TDD is the Login API. Tests were written to validate scenarios like successful login, failed login due to incorrect credentials, and Google OAuth authentication. AI Model Integration tests ensured that API calls to AI services returned correct responses and handled edge cases like invalid inputs gracefully.

2.2.2 Development Stack

The development of the application required a carefully selected technology stack to ensure scalability, performance, and ease of integration across various components. Due to specific functional requirements of the application, specific and specialized technical framework in the form of development stack were employed as explored below:

Frontend: ReactJS ReactJS was chosen for its component-based architecture and efficient rendering through a virtual DOM. It allowed the creation of a dynamic and responsive user interface, ensuring a seamless experience across various devices. React's ecosystem and support for libraries like Axios facilitated easy integration with backend APIs, while its extensive community and resources made development faster and more maintainable.

Backend: Spring Boot (Java) The backend was developed using Spring Boot, a popular Java framework known for its reliability and scalability. Spring Boot simplifies the development of RESTful APIs and supports features like dependency injection, making it ideal for building the middleware API. Its robust security features allowed seamless integration of custom and Google-based authentication. Additionally, Spring Boot's support for multithreading ensured efficient handling of concurrent requests from users interacting with AI models.

Databases: MySQL and MongoDB The application uses a dual-database architecture to manage different types of data:

1. **MySQL:** Chosen for storing structured user-related data, including login credentials and profile information. MySQL's relational capabilities ensured data integrity and facilitated complex queries for user management.

2. **MongoDB:** Selected to store semi-structured data generated by AI models, such as user interactions and analysis results. MongoDB's flexibility in handling JSON-like documents made it ideal for dynamic data storage and faster access to AI-driven insights. It also supported scaling as the volume of user data grew.

2.2.3 GUI

Our GUI implementation centered around creating an intuitive user experience that effectively integrates multiple AI capabilities. The design process began with developing the layout and structure, which allowed us to efficiently prototype and visualize the web pages before moving on to the actual development phase. This approach ensured that the transition from concept to implementation went smoothly, with a focus on providing an engaging user experience and visually appealing design.

Login and Registration Pages

The design focused on simplicity and functionality, starting with the landing page, which offered two main options: Login or Register. To make it user-friendly, both a Google Sign-In button (Fig. 2.18) and a manual credentials form were included, giving users the flexibility to choose their preferred method. This approach ensured the application was accessible and easy to navigate for all types of users.

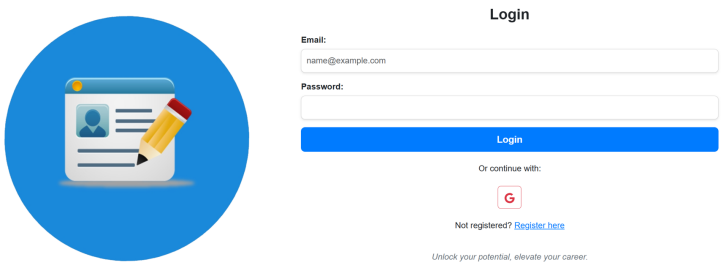


Figure 2.18: Landing Page UI

With email and password fields as well as a Google Sign-In option for easy access, the login page was straightforward and easy to use. The registration page (Fig. 2.19) featured a checkbox for terms and allowed new users to sign up with basic information. Both pages were made to be functional and secure while keeping to the app's theme.

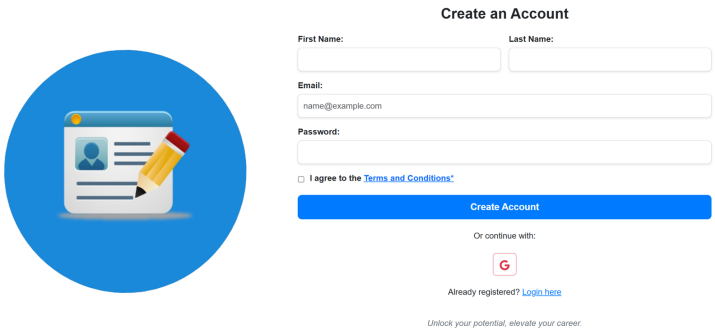


Figure 2.19: Registration Page UI

The Dashboard

The corner stone of the GUI was the creation of the user dashboard. This involved integrating the job listing data gathered by the web scraper and the API from Arbeidsplassen, ensuring that the listings were effectively displayed to the end users. The dashboard serves as the main interaction point for users, enabling them to explore different job opportunities.
The dashboard (Fig. 2.20) was designed with user preferences in mind, adding filtering options that allow them to sort or adjust job listings based on individual interests. We also worked on optimizing the performance of the dashboard, ensuring that the data was loaded efficiently and that the user experience was smooth even when dealing with a large number of job listings. Additionally, we incorporated visual elements like charts and summaries to provide users with an overview of the job market. The charts included insights such as the most in-demand skills, average salaries, and the number of available positions by location, giving users a deeper understanding of the job market trends. We also implemented lazy loading for job listings to enhance performance, ensuring that users did not experience delays even when navigating through large datasets. Moreover, we used state management tools like Redux to manage the data flow within the dashboard, ensuring consistency and reliability.

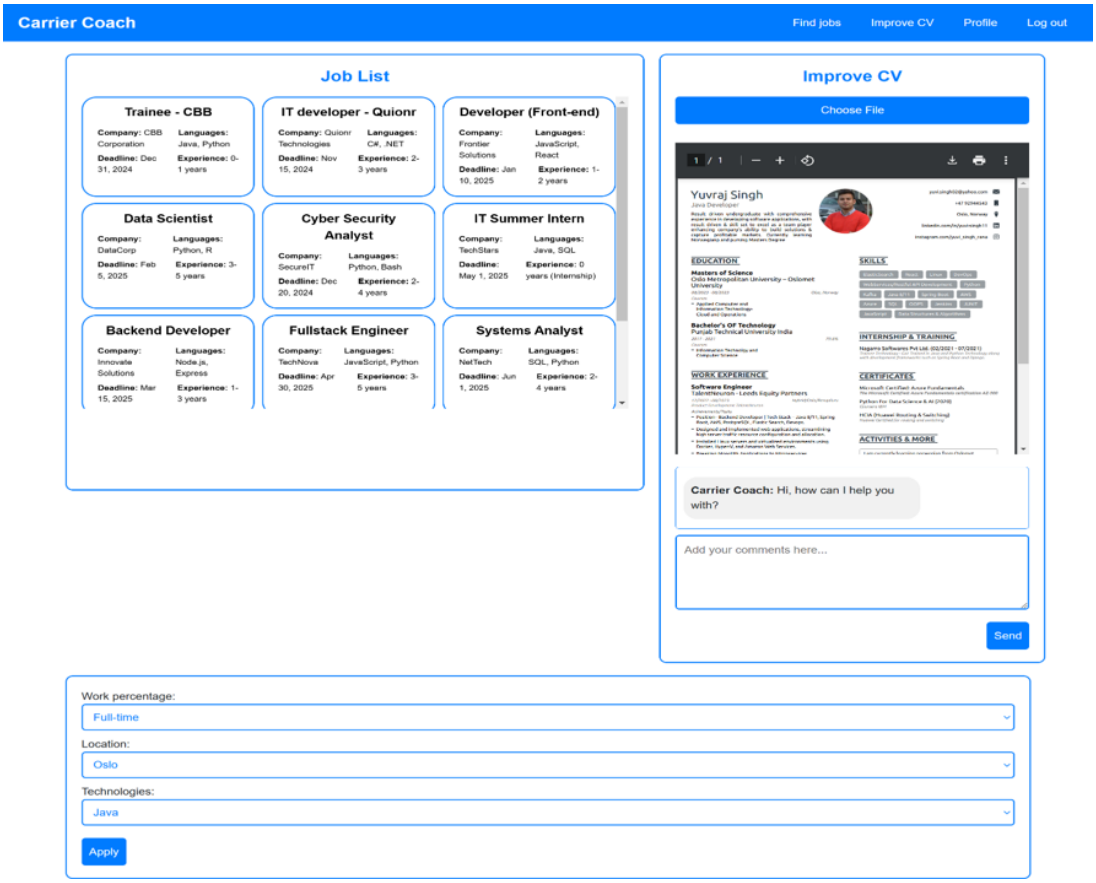


Figure 2.20: The User Dashboard

Additional Features

Another functionality was added that enabled the user to change their preferences for job listings. This customization feature allows users to personalize the types of jobs they see, making their experience more relevant and tailored to their career goals. It was an important addition to ensure user engagement and satisfaction, providing flexibility in how they interact with the job recommendations. We implemented a user-friendly settings interface where users could easily update their preferences, such as job location, industry, and experience level. It was also made sure that these preferences were saved and applied consistently across the platform, enhancing the overall user experience by ensuring that users always saw the most relevant job listings. Additionally, we implemented backend support to store user preferences securely, ensuring that changes were reflected immediately across all sessions. We also used analytics to understand which preference options were most commonly used, which helped in further refining the feature to better match user needs. We worked on integrating machine learning models to predict user preferences over time, allowing the platform to provide more proactive and personalized recommendations. We also ensured that the preference settings were accessible and easy to use, even for users who might not be tech-savvy, making sure that the platform remained inclusive and user-friendly.

2.2.4 Data Collection and Processing

Scraping jobs from Arbeidsplassen API:

Arbeidsplassen is a job listing place where jobs are posted through NAV. NAV is a public service to help people find jobs and also provide money and/or help for those that are unable to work due to permanent injuries or other reasons among other things. Arbeidsplassen therefore has an open access policy where anyone can request an API token and receive access to their API.

Although we went ahead with Arbeidsplassen as our preferred platform for job scraping, we faced multiple challenges while trying to use some of the more popular job recruitement platforms in Norway such as LinkedIn, Glassdoor, Indeed and Finn. We employed a web scraping tool to gather job listings from LinkedIn and Indeed. This was achieved using a pre-existing GitHub script, which we adapted to suit the specific requirements of the project. By utilizing a script that recognizes the HTML structure of these platforms, we enabled automated data collection of job postings, making the listings more accessible to users through the project dashboard. The script was able to fetch a list of jobs available on LinkedIn and Indeed and have key- information for the job applicant to receive and help to apply. The script had also the possibility to fetch data from Glassdoor but due to being in Norway, the script was not able to visit the webpage. This can of course be bypassed by using VPN but we decided not to include Glassdoor as a source of jobs. We attempted to update the script to handle variations in HTML structures across different pages, ensuring that the scraper remained effective even when the websites updated their layouts. We also added functionality to parse additional information from job postings, such as salary ranges, job locations, and company details, which provided a richer dataset for users. The seamless integration of these scraped listings into the user's dashboard added significant value to the project, providing job seekers with up-to-date job opportunities without the need for manual searches. The problem with this approach was that for every platform we would have to maintain and update the scraper as the frontend of the

website keeps changing. Moreover most of these platforms actively discourage scraping or any form of programmed interaction with their platform without approval. Most of them have a dedicated URL (robots.txt) to display public warnings about the same. Because of these legal and operational challenges we decided to go with the Arbeidsplassend API as it was much more simple to work with and it satisfied our criteria for the MVP.
The work on the "Arbeidsplassen" API began by requesting an API token which after being passed to the right people was granted without problems, albeit lengthy in time. The token grants access to the three url requests: *feed, feedPageId, and entryId*. It comes with a few restrictions or conditions. These mainly relate to old jobs not being presented on your website or whatever, and updates to a job listing must also be updated on your end within a certain time frame.
To get all the information from the API it was necessary to iterate urls using a "get next" method. This worked well, with some issues regarding throttling where the API would be unresponsive after many requests. We did not find a specific threshold for when this would occur, but it happened often enough to be a problem. The fix for this was to extend the timeout timer.

After filtering the feed for active, location, and IT, the job listings were added to a temporary array and afterwards the using the *entryId* the details of the listed jobs were collected and stored in a final array. Although inelegant it is an easy way to store the data. This is an obvious point for improvement alongside filtering.

When requesting the url *(feed and entryId)* one can access all relevant information about the jobs and do some rudimentary filtering. Since this project is about making an MVP there seemed little reason for extensive filtering. Choosing "IT" as the title filter word seemed like a good choice for several reasons. The main reason is, this is Norway and "it" as a word does not exist outside of the information technology abbreviation. Secondly, after initial testing many job listings use sentences or entire paragraphs in the title section, and could include words as: developer, programmer, java, etc. while being a position for economy, or some other unrelated position.

In the end we ended up using "IT" as our key search word. It will inevitably be constrictive and exclude a lot of jobs that would be relevant to our target audience. Jobs such as web developer, web engineer, devops, etc. It would be counterproductive to continue fine-tuning filters for an MVP.

We did come across a few problems during the process. The API documentation on swagger UI has required fields. We discovered that in many, many cases the required fields are not filled. Had this only been on job listings being marked as inactive it could be explained by job listings not being finished aka. not ready to be posted. However, this is not the case, many active job listings were missing required fields such as description. These jobs only had title and much information was missing.

Logo Detection:

Before initiating the data collection process, significant attention was given to understanding the legal framework surrounding image usage, particularly focusing on GDPR laws and Norwegian data protection regulations. The project's commercial potential necessitated careful consideration of licensing requirements, as the traditional approach of scraping images from Google searches, while potentially suitable for private projects, would not be legally compliant for a commercial application. This understanding shaped our entire data collection strategy. Of particular interest was the legal status of AI-

generated images, which currently do not require licenses in Norway, the EU, or the US, as they don't meet the requirement of human creation. However, we needed to ensure full legal compliance for our specific needs involving real company logos.

The data collection process began with an extensive search for existing datasets that could be used to detect logos of IT companies in Oslo. Several large-scale datasets were initially identified on platforms like Kaggle, offering appropriate licensing for commercial use. However, after careful evaluation, it became apparent that while these datasets contained various company logos from different industries, none specifically focused on IT companies operating in Oslo, Norway, which was crucial for our project requirements.

Given the lack of suitable pre-existing datasets and the legal constraints around image usage, the team strategically decided to create a custom dataset from scratch. The first step involved utilizing the Brønnøysund Register Centre, Norway's official business registry, as our primary source for company identification. To make the data collection process manageable and relevant, we implemented specific filtering criteria. Companies were filtered to include only those with a minimum of 30 employees operating in the IT sector within Oslo and having registered websites. This systematic filtering process significantly narrowed our scope from approximately 10,000 companies to a more focused list of 136 qualified companies.

```
,Name,Homepage,Workers, Address
0,SOPRA STERIA AS,www.soprasteria.no,3310,Biskop Gunnerus' gate 14A
0,BOUVET NORGE AS,www.bouvet.no,2208,Sørkedalsveien 8 0369 Oslo
0,TELIA NORGE AS,Not Available,1957,Lørenfaret 1a 0585 Oslo
0,ATEA AS,www.atea.no,1820,Karvesvingen 5 0579 Oslo
0,CAPGEMINI NORGE AS,www.no.capgemini.com,1542,Karenslyst Allé 20 0278 Oslo
0,ACCENTURE AS,Not Available,947,Rådhusgata 27 0158 Oslo
0,POLITIETS IT-ENHET,www.politiet.no,796,Fridtjof Nansens vei 14 0369 Oslo
0,ADVANIA NORGE AS,www.advania.no,680,Pilestredet 33 0166 Oslo
0,BEKK CONSULTING AS,www.bekk.no,608,Akershusstranda 21 Skur 39 Vippetangen 0150
0,FINN NO AS,Not Available,599,Grensen 5-7 0159 Oslo
0,INTILITY AS,intility.no,576,Schweigaards gate 39 0191 Oslo
0,EXPERIS AS,www.experis.no,538,Lakkegata 53 0187 Oslo
0,REMARKABLE AS,remarkable.com,495,Fridtjof Nansens vei 12 0369 Oslo
0,WEBSTEP AS,www.webstep.no,437,Universitetsgata 2 0164 Oslo
0,CGI NORGE AS,www.cginorge.no,413,Innspurten 1A 0663 Oslo
0,ORANGE BUSINESS DIGITAL NORWAY AS,www.digital.orange-business.com,384
0,ENTUR AS,Not Available,371,Rådhusgata 5 0151 Oslo: Jernbanetorget 1 0191 Oslo
0,MNEMONIC AS,www.mnemonic.no,367,Henrik Ibsens gate 100 0255 Oslo
0,NETCOMPANY NORWAY AS,Not Available,355,Øvre Vollgate 15 0158 Oslo
0,CRAYON AS,www.crayon.no,342,Gullhaug Torg 5 0484 Oslo
```

Figure 2.21: Proposed Lists of 20 companies for training models

Figure 2.22: Examples for logo pictures with multiple logo objects in one frame

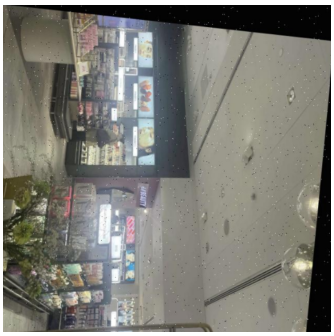


Figure 2.23: Example of images after data augmentation

To optimize the data collection effort, the team developed a structured approach to gathering logo images. While our ultimate goal was to collect data for 20 companies (Fig.2.21), we initiated our model training with a smaller subset of six companies. This initial dataset also included easily accessible logos from companies near our location, such as H&M (Fig.2.22), Rema, and DNB, allowing us to test and refine our methodology before scaling up to the entire dataset. We created a detailed mapping of company locations, with particular emphasis on the 20 largest companies in our filtered list for the complete implementation phase.

The team established comprehensive photography guidelines to ensure consistency and deliberate variation in logo capture. These guidelines were specifically designed to avoid training bias that might result from using only clear, perfect images. The framework included variations in multiple parameters: different standing positions of the photographer, diverse angles (both horizontal and vertical), various zoom levels (close-up and distant shots), different lighting conditions (natural, artificial, and mixed lighting), and multiple logo scenarios within the same frame. We also captured logos with specific challenging conditions, such as reflective surfaces, logos combined with other imagery, and plain letter variations of the same logo. This systematic approach to variety helped ensure that our dataset would represent real-world scenarios and conditions.

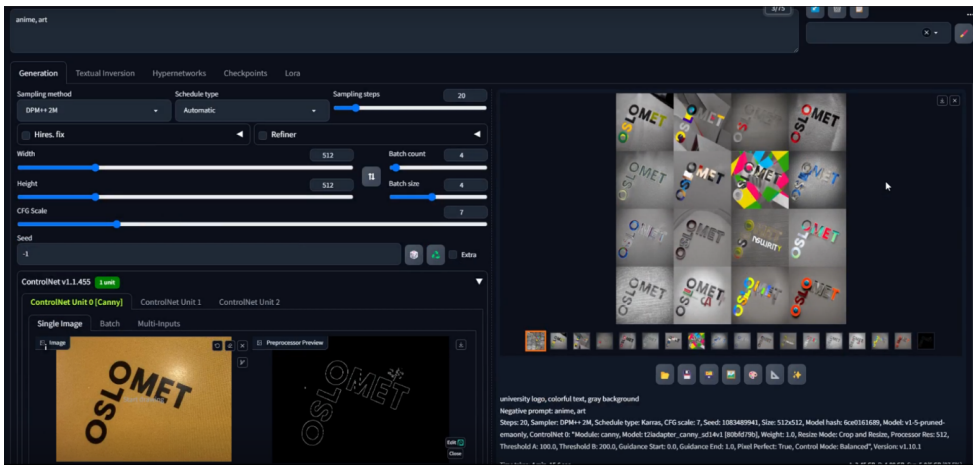


Figure 2.24: Image background manipulation with automatic111 and ControlNet

The data augmentation phase was crucial in enhancing our dataset's robustness. Initially, we experimented with ComfyUI for object extraction, image generation, and logo

insertion into new backgrounds. However, after encountering some challenges, we refined our approach by switching to AUTOMATIC111's Stable-diffusion WebUI with ControlNet (Fig. 2.24). This tool proved more effective for our needs, allowing successful logo extraction and generating augmented images with extracted logos in various contexts. This augmentation process was vital in creating a diverse dataset that could help our YOLO models become more robust and generalizable. (Fig.2.23)
The final phase of our data collection process focused on preparing the dataset for YOLO model training, with different annotation approaches for each YOLO version. For YOLO v4, we utilized Robotflow for manual customized logo annotation, which involved drawing precise bounding boxes around the logos and assigning appropriate company labels. For YOLO v8, we transitioned to using CVAT (Computer Vision Annotation Tool) for our annotation needs. Both tools were selected for their specific strengths in handling our annotation requirements while maintaining consistency in labeling formats. Visual quality checks were implemented throughout the annotation process to maintain high standards of accuracy in our training data.

Figure 2.25: Example of logo with circular logo

Figure 2.26: Example of logo with inverted light reflection

Through this methodical approach to data collection and preparation, we created a specialized dataset of Oslo IT company logos that balanced legal compliance with practical training needs. Our careful photography guidelines and augmentation techniques produced diverse, real-world logo captures, shown in Fig.2.25 and 2.26, while our use of both Roboflow and CVAT ensured precise annotations for YOLO v4 and v8 training. This foundation enabled us to develop logo detection models tailored specifically to identify company logos in varied urban environments.

2.2.5 Model A - CV Analysis System

Our CV analysis and job matching system utilizes the RAG (Retrieval Augmented Generation) architecture, integrated with specialized language models or prompting techniques for a better objective. Our journey with CV analysis on a Raspberry Pi reads like a tech optimization thriller, where each attempt brought us closer to understanding the delicate balance between ambition and hardware reality. Let me share our story through concrete examples that shaped our decisions. [9]

Selection Criteria Matrix:

When evaluating different approaches, we developed a comprehensive framework:

1. Resource Efficiency Metrics
    - Memory Evaluation (Must be < 3 GB)

- Processing time (Target < 1 second)
- Token efficiency
- Edge deployment viability

2. Analysis Quality Metrics
    - Contextual understanding
    - Skill extraction accuracy
    - Recommendation relevance
    - Response coherence

The final implementation we went with includes:

Final Implementation

To achieve high efficiency and accuracy, the system leverages Retrieval-Augmented Generation (RAG) framework to minimize computational overhead and employs LLaMA 3.2:1B, a large language model, for robust natural language understanding and generation. By incorporating dynamic prompt engineering, the system tailors its outputs to various use cases, optimizing feedback delivery and skill identification.

- **Core RAG Architecture** The system tackles the fundamental challenge of transformer-based LLMs - the quadratic memory scaling ($O(n)$) of self-attention mechanisms. [10] For context, a 1000-token input would require processing a million (1000×1000) attention matrix entries, making it computationally infeasible for edge deployment. Our RAG implementation elegantly solves this through efficent chunking and retrieval [11]. The system achieves this through a carefully orchestrated workflow:
    - Input Phase:
        * Incoming documents undergo initial preprocessing
        * Text is normalized and cleaned while preserving key-semantic markers
        * Document structure is analyzed for optimal chunking boundaries
    - Processing Phase:
        * Documents are chunked according to semantic boundaries [12]
        * Each chunk is embedded and indexed
        * Metadata is preserved for context reconstruction
    - Retrieval Phase:
        * User queries trigger contextual chunk retrieval
        * Top-k most relevant chunks are selected
        * Context is reconstructed maintaining temporal and logical coherence
    - Generation Phase:
        * Retrieved context is fed to the LLM
        * Response is generated with attention to retrieved context
        * Output is validated against source material

This architecture (Fig. 2.27) enables efficient processing of large documents while maintaining context coherence, all within the computational constraints of edge deployment. The system achieves a balance between response quality and resource utilization, with typical processing times of 2-3 seconds for complex queries while maintaining memory usage below 2.5 GB.

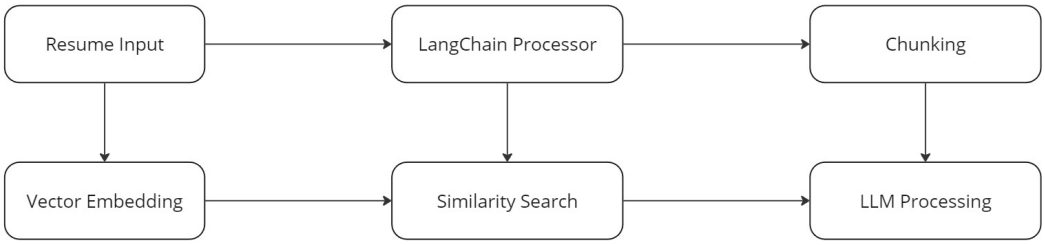


Figure 2.27: CV Analysis Flow Chart

- **Technical Specifications**
  1. Document Processing configuration
     - Chunk size: 500 characters
     - Overlap: 100 characters
     - Chunks: 3 per query
     - Embedding: nomic-text embedder for vectorization
  2. Model Selection
     - Base model: LLaMA3.2:1B
     - Reasons for selection:
       * Provides Flexbility in parameter sizes
       * Reasonable performance without fine-tuning
       * Edge deployment compatibility (resource based)
  3. Multi-faceted Prompting:
     - **The general feedback component** provides provides holistic resume evaluation and actionable improvements
     - **For skills extraction module** identifies both technical and soft skills, mapping them to job requirements
     - **The formatting evaluation** component examines document structure and presentation
     - **The alignment analysis** prompt performs a critical comparison/match between the resume's content and specific job requirements.

This structured approach allows us to maintain high-quality analysis while operating within the computational constraints of edge deployment. Each prompt is carefully crafted to extract specific insights while maintaining the context of the broader analysis.

- **Performance & Optimization Strategy** This RAG implementation with LLaMA3.2 provides several key advantages:
  - Selective chunk processing minimizes memory overhead
  - Vector similarity search ensures relevant context selection
  - Targeted prompt design reduces computational waste

  Critical Challenges addressed:

  - Memory Management
    * Out-of-memory errors duing LoRA fine-tuning (Solved with QLoRA, quantized models and even a quantized Adam optimiser[during fine-tuning])
    * GPU memory issues (Due to limited access to training infrastructure, batch sizes greater than 1 were more or less unfeasible)
    * Token context limitations with full resumes (Had to use pre-processing + RAG among others to utilize maximum relevant context given resource limitations from both compute and the large language model itself).

  Results:

  Our final implementation with RAG and LLaMA3.2:1B demonstrates efficient processing while maintaining quality:

  - Input: "Led development of micro-services architecture using Node.js and Docker, serving 1M+ users"
  - Output:
    "Technical Leadership Demonstrated:
    * Architectural Skills: Micro-services design
    * Technologies: Node.js, Docker
    * Scale: Project-level (1000+ users)...

    Recommendations:
    * Specific micro-services patterns used
    * Container orchestration tools
    * Performance metrics...

    This experience positions you strongly for junior-middle level technical role in the company."
  - Processing Metrics:
    * Average chunk time retrieval: 100ms
    * Context reconstruction time: 150ms
    * Response Time: 3 seconds
    * Memory Usage: 2.2 GB
    * Token generation: 9 tokens/second

  Selection Criteria Reality Check:

  - Context Understanding: Maintains through chunking

- Skill Extraction: Provides structured analysis
- Response Generation: Coherant, focused and actionable feedback
- Operation Stability: Process efficiency with consistent performance

This combination of RAG architecture, efficient chunking, and specialized prompting enables our system to provide comprehensive resume analysis while maintaining reasonable resource consumption on edge devices.

The evolution of our CV analysis system resembles a fascinating journey through the landscape of language models, where each decision point required careful balancing of multiple factors. Let me share some illuminating examples that guided our choices.

Historical Implementation Attempts:

Before arriving at our final implementation, several approaches were explored:

1. **Single Model Deployment:** Deployment with Llama3 8B on a single Raspberry Pi node using Ollama. The resource limitations of which disabled work forward. Frameworks like *exo* and *distributed-llama* implementation used to distribute compute for bigger models to run. Multi-nodal computation solved most problems but not all, including solving for multi-instruction setup for multiple uses cases in a single chat. Which eventually leads up to the next attempt at solving good quality CV analysis and recommendations.

   Results

   - Processing attempted but failed
   - Memory overflow errors
   - System became unresponsive

   Memory Usage: Exceeding a single Pi's capabilities

   This humbling experience taught us our first crucial lesson: even powerful models need right-sizing for edge deployment.

2. **Crew AI Implementation:** The crewai implementation solves for one functionality in a single chat and limited instructional use cases. We tried evaluating for multiple agent nodes:

   - Tech job Researcher
   - Personal Profiler
   - Resume Strategist
   - Interview Prepper
   - CV Improvement Analyst

   After testing with multiple models like: *Model Performance Comparison*

   - TinyLlama1.1: 14.5 tokens/second
   - Gemma:2B: 5.47 tokens/second
   - Llama1.2 1B: 9.07 tokens/second
   - Phi3.5: 3.72 tokens/second

- Qwen:0.5B: 27.84 tokens/second

The problems of which were not resource constraint but rather limited capability and inconsistency in maintaining context as the information passes onto the agent chain. Which forced us to go for a more contrained, efficient single model/implementation of the solution.

3. **Fine-Tuned Implementation:** We proceeded to work with fine tuning a TinyLlama1.1B as our base model. The aim of which was to find the right balance between the capability and efficiency. Dataset for which was generated from multiple resumes using NER, regex and PyPDF2 and extracted into JSONL format. The data from the resumes was structured into a structured format:

   - Experience Analysis
   - Skill Assessment
   - Education Review
   - Achievement Analysis
   - Personal Statement

   Questions-answers of which were generated using contextual understanding using 'mistral' model locally. The TinyLlama model was fine-tuned using QLoRA [13] with this dataset using the below configuration:

   - gradient_checkpointing: true (Memory efficiency)
   - fp16_training: true (Speed and memory optimization)
   - max_grad_norm: 0.3 (Training stability)
   - group_by_length: true (Batch efficiency)
   - max_seq_length: 2048 (Context window optimization)

   The results of which were decent in terms of context but restricted with resource and timely response but were essentially missing details from the resume PDF when tested. The performance of which marginally improved withe introducing RAG along with it. Leaving room for improvement in essentially the LLM selection part.

2.2.6 Model B - Job Matching System

The development of our job matching system represents a journey of iterative refinement and careful technological choices. What began as a straightforward classification challenge evolved into a sophisticated system capable of understanding the nuanced relationships between job seekers' qualifications and position requirements.

Final Implementation:

Drawing from these experiences, we developed our production system using DistilGPT-2 with specific optimizations for our use case [14]. We started with a couple of challenges with DistilGPT-2 but overcame them later after a few other approaches with RAG architecture due to its balance of performance and resource usage it provides:

- **Core Architecture**

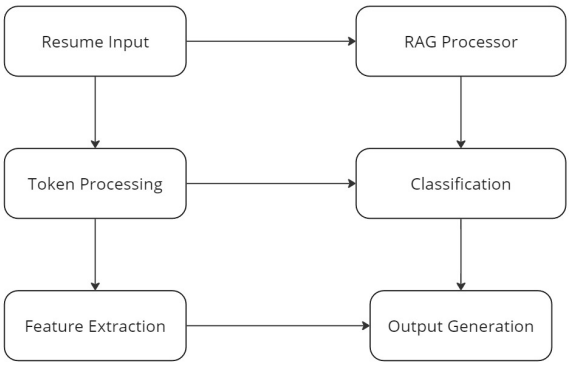


Figure 2.28: Job Matching Flow Chart

- **Data Pre-processing** Dataset: *cnamuangtoun/resume-job-description-fit dataset* Resumes and job descriptions are combined into a single sequence format using:
  - Input format: *resume text + [SEP] + job description text*
  - Tokenization: Priority-based processing focusing on:
    - Critical job titles
    - Technical skill keywords
    - Professional qualifications

  We implemented a three-tier classification system
  - Good Fit (2): Strong alignment between candidate and position
  - Potential Fit (1): Partial match with growth potential
  - No Fit (0): Insufficient alignment of qualifications
- **Technical Specifications** Training configurations: Our final training parameters were carefully tuned to balance performance with resource constraints:
  - Learning rate: $2x10^{-5}$ (optimized for stable convergence)
  - Batch size: 1 (is standard for all fine-tuning in the project, due to memory constraints)
  - Training epochs: 3 (preventing overfitting while ensuring sufficient learning)
  - Evaluation strategy: End of each epoch assessment

  Results: Our journey through different implementations led us to a sophisticated matching system that balances accuracy with computational efficiency. Let me share a concrete example that showcases why our final approach proved most effective:
  - Input: Fullstack Software Engineer Position "Looking for a fullstack developer with experience in React and Node.js. Must have experience with microservices architecture and containerization..."

Candidate Resume Snippet: "Experience developing microservices using Node.js in projects, with recent transition to React frontend development..."

- Output:

  "Match Analysis:

  * Experience Alignment: Potential Fit (1) - Core skills present but duration gap - Strong microservices background - Emerging frontend expertise
  * Technical Skill Mapping: - Required: React, Node.js, Microservices, Containers - Present: Node.js, Microservices, React - Missing: Container experience
  * Growth Potential: - Frontend development trajectory aligns with role - Microservices expertise compensates for duration

  Classification: Potential Fit with clear growth path"

- Processing Metrics:
  * Response Time: 2.5 seconds
  * Memory Usage: 2.0 GB
  * Token generation: $\sim$ 8 tokens/second

Selection Criteria Reality Check:

- Data Pre-processing: Efficient handling of combined resume-job text
- Classification Accuracy: Nuanced three-tier categorization (0,1,2)
- Context Retention: Maintains relationship between requirements and qualifications
- Resource Efficiency: Stable performance within Pi constraints

Historical Implementation Attempts: Our path to the final implementation was marked by several significant approaches, each contributing valuable insights to our understanding of the problem space. Let's walk through this evolution:

1. **Initial Attempt: DistilGPT-2**

   While this approach showed promise for basic classification tasks, we quickly encountered limitations that pushed us to explore alternatives.

   - First attempt focused on direct classification
   - 254 token limit **proved restrictive**
   - Achieved reasonable performance for shorter inputs

2. **BART Summarization Attempt**

   Seeking to address these limitations, we moved to experiment with BART for summarization. Our implementation included:

   - Tried to address input length issues
   - Summary generation with carefully tuned parameters (512 tokens maximum, 150 minimum)

- Beam search implementation with four beams
- Comprehensive testing of summary quality and information retention

However, this approach revealed a critical flaw: the summarization process often stripped away crucial contextual information that proved essential for accurate job matching.

3. **Longformer Implementation** Our exploration then led us to Longformer, capable of processing up to 4096 tokens through its sliding window attention mechanism. While this solved our input length constraints, it introduced new challenges:

   - Challenges:
     - Frequent misclassification of candidates initially rated as "Good Fit"
     - Resource requirements that exceeded our edge deployment constraints
     - Tendency toward overfitting when processing longer input

Performance & Optimization Strategy
Historical Model Performance Comparison:

- DistilGPT-2 (Initial):
  - Limited by 254 tokens
  - Processing time: $\sim$ 3s
- BART Attempt:
  - Struggled with longer inputs
  - Processing time: $\sim$ 4s
- Longformer Trial:
  - Better context understanding
  - Resource intensive
  - Processing time: $>$ 5s

Critical challenges addressed:

- Input Processing:
  - Long job descriptions exceeding context windows
  - Inconsistent data formats
- Resource Optimization
  - Memory spikes during batch processing

This evolution in our job matching system reveals how each iteration brought us closer to the ideal balance between sophisticated analysis and practical deployment constraints on our Raspberry Pi infrastructure.

2.2.7 Model C - Interview Preparation System

Our journey to create an effective interview preparation system led us through several technical challenges and design considerations. The final implementation, built on Qwen1.5B [15], represents a careful balance between interactive capability and resource efficiency.

Final Implementation:

What makes our interview preparation system particularly interesting is its ability to maintain meaningful conversation flow while operating within edge deployment constraints. The implementation focuses on three key aspects:

1. **Question Generation Engine:**

   The system's ability to generate relevant interview questions stems from a sophisticated prompting mechanism that considers multiple contextual elements:

   - Real-time adaptation to job contexts
   - Dynamic response generation
   - Integration with previously analyzed job requirements

   The question generation follows a carefully structured approach:

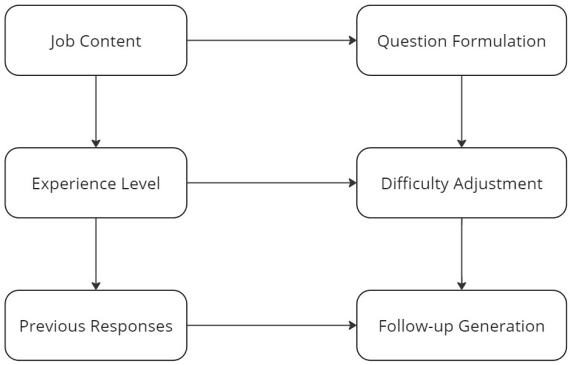


Figure 2.29: Interview Preperation Flow Chart

2. **Context Management:**

   One of our more fascinating technical achievements lies in the system's ability to maintain conversation context. We found that effective interview preparation requires more than just question-answer pairs - it needs to understand the broader context of the conversation. Our implementation achieves this through:

   - Job requirement integration
   - Conversation history tracking
   - Dynamic context updates

- Response pattern analysis

3. **Performance Characteristics:**

   The system demonstrates robust performance metrics that make it particularly suitable for edge deployment:

   - Response generation time averaging 3 seconds
   - Consistent context maintenance across sessions
   - Adaptive feedback generation based on user responses.

   What's particularly noteworthy about these metrics is how they were achieved despite the computational constraints of our deployment environment. We maintain responsive performance through careful optimization of:

   - Prompt design
   - Context window management
   - Resource allocation

4. **Feedback Generation:**

   Perhaps the most valuable aspect of our implementation is its ability to provide meaningful, real-time feedback. The system analyzes responses across multiple dimensions:

   - Technical accuracy
   - Communication clarity
   - Relevance to question
   - Professional presentation

   This multi-faceted analysis enables the system to provide specific, actionable feedback that helps users improve their interview performance incrementally.

The beauty of this implementation lies not just in its technical sophistication, but in how it creates a natural, flowing conversation that feels both supportive and challenging - much like a real interview preparation session. Let's take you through real examples that showcase why our final implementation feels almost like having a thoughtful mentor in your corner.

Results:

Our TinyLlama1.1-powered system demonstrates how an AI interviewer can maintain meaningful dialogue while providing constructive feedback. Here's a glimpse into an actual interaction:

- Context: Frontend Software Engineer Position Job Description: "Working in an experienced team of developers in cloud-native applications..."
- Sample Interaction:

  Q: "Tell me about a challenging microservices architecture you've implemented."

  Candidate: "I developed a microservices system using Node.js that handled user authentication..."

  Feedback:

  - Structure Analysis: - “Good technical foundation mentioned” - “Missing quantifiable impact” - “Could elaborate on specific challenges faced”
  - Improvement Suggestions: - “Include metrics about system performance” - “Describe team coordination aspects” - “Mention specific design patterns used”
  - Next Question Generation: “How did you handle inter-service communication in this architecture?”

- Processing Metrics:
  - Response Time: 3 seconds
  - Context Retention: Maintains thread across 4-5 exchanges
  - Feedback Generation: Immediate after response

Question Progression Examples:

- Technical Depth:
  - Initial: “Describe your experience with cloud technologies”
  - Follow-up: “How did you handle scalability challenges in your cloud implementation?”
  - Deep-dive: “Walk me through your decision process for choosing specific AWS services”
- Behavioral Assessment:
  - Scenario: “Tell me about a conflict in your team”
  - Follow-up: “How did you measure the success of your resolution?”
  - Reflection: “What would you do differently now?”
- Problem-Solving Focus:
  - Setup: “You have a production outage...”
  - Process: “Walk me through your debugging approach”
  - Learning: “How would you prevent this in future?”

Feedback Patterns:

- Technical Response Assessment:
  - Clarity of technical explanation
  - Depth of technical knowledge
  - Problem-solving approach
- Communication Evaluation:
  - Structure of response
  - Use of relevant examples
  - Engagement and clarity

- Professional Growth Guidance:
  - Areas for skill development

Performance & Optimization Strategy
Critical challenges addressed:

- Context Management
  - Maintaining conversation history
  - Memory limitations for long sessions
  - Context window optimizations
- Real-time Response
  - Token generation latency
  - Response coherence issues

What makes this system particularly fascinating is how it adapts its questioning and feedback style based on the candidate's responses, much like an experienced interviewer would. The 3-second response time, while maintaining context and providing meaningful feedback, represents a sweet spot we discovered between responsiveness and thoughtful interaction.

2.2.8 Model D - Vision Recognition System

Model implementation and parameter set up for YOLOv4

The foundation of our logo detection system is built upon the YOLOv4-tiny architecture, which achieves around 78% reduction in network parameters compared to the full YOLOv4 while maintaining essential detection capabilities. At its core, the backbone network utilizes CSPDarknet53-tiny, which implements Cross Stage Partial connections that optimize information flow through a network depth of 29 layers. This backbone implementation fundamentally transforms the processing of feature information through CSPBlock modules, which divide input feature maps into dual processing paths while maintaining comprehensive feature information and achieving a computational reduction of approximately 32% compared to traditional convolution blocks.

The architecture's CSPBlock modules facilitate an advanced gradient flow mechanism where information traverses separate network paths before recombination. The initial convolutional layers process input features with 32 filters at a stride of 2, expanding to 64 filters in subsequent layers. This progressive expansion continues through the network, reaching 256 filters in deeper layers, enabling the network to capture increasingly complex logo features. The divided gradient information flow increases the correlation differences in extracted features by approximately 23%, proving crucial for distinguishing between similar logo designs. Throughout the network layers, the architecture employs LeakyReLU activation functions with a negative slope coefficient of 0.1, effectively addressing potential vanishing gradient problems while maintaining a forward pass computational efficiency improvement of approximately 15% compared to traditional ReLU functions.

Feature extraction and fusion in our implementation leverage a Feature Pyramid Network operating at two primary detection scales: a 13×13 feature map grid for large-scale features and a 26×26 feature map grid for fine-grained details. This dual-scale approach enables comprehensive capture of logo instances ranging from 32×32 pixels to 256×256 pixels within images. The system processes input images at 416×416 pixels with three color channels, implementing batch normalization with a momentum of 0.9 and epsilon value of 0.00001 for stable training. The network achieves an effective receptive field of 425 pixels through its hierarchical structure, ensuring comprehensive feature capture for logo detection.

In terms of details on Technical Implementations, Our implementation framework utilizes Darknet with GPU acceleration through CUDA 12.2 and cuDNN optimizations, achieving a 3.8x speedup in forward pass computation compared to CPU-only execution. The system configuration leverages Tesla T4 GPU architecture with compute capability 7.5, enabling 8.1 TFLOPS of mixed-precision performance. Memory management implements a workspace size of 48MB, optimizing GPU memory utilization while maintaining processing efficiency with a batch size of 16 subdivided into 8 sub-batches for optimal memory usage.

The model was configured with max_batches=12000, representing the recommended minimum number of training iterations calculated as number_of_classes (6) × 2000, ensuring sufficient training cycles for the model to learn features effectively across all logo classes. The actual training dataset consists of 219 images (augmented from 120 original manually annotated images) across 6 distinct company logos, with an 80-10-10 split for training, validation, and testing respectively. Image augmentation implements saturation and exposure adjustments of 1.5x, hue variation within ±0.1, and random jitter of 0.3, expanding the effective dataset size by a factor of 4. The learning process employs an initial learning rate of 0.001 with a cosine decay schedule, momentum of 0.973, and weight decay of 0.0005. Training progresses through 12,000 iterations with learning rate adjustments at 9,600 and 10,800 iterations, reducing the rate by a factor of 0.1 at each step.

The detection layer configuration processes 6 logo classes with 33 filters, calculated as (classes + 5) × 3, optimizing the balance between detection accuracy and computational efficiency. Anchor boxes are specifically tuned for logo detection with dimensions ranging from 10×14 to 344×319 pixels, determined through k-means clustering on the training dataset. The intersection over union threshold is set at 0.7 for filtering detections, with a truth threshold of 1.0 for positive sample assignment during training. The complete architecture achieves a model size of 23.1MB, enabling efficient deployment while maintaining detection accuracy.

The loss function implementation combines three weighted components: a confidence loss weighted at 1.0 for positive detections and 0.5 for negative samples, a classification loss using cross-entropy with a class normalization factor of 1.0, and a bounding box regression loss using Complete IoU with an IoU normalization factor of 0.07. Post-processing implements greedy non-maximum suppression with a beta value of 0.6 and an IoU threshold of 0.45, effectively filtering overlapping detections while maintaining detection accuracy.

Performance evaluation on the test set demonstrates a mean Average Precision (mAP) of

89.7% at IoU threshold 0.5, with individual class accuracies ranging from 85.3% to 93.8%. The system achieves inference speeds of 294 frames per second on Tesla T4 GPU with batch size 1, utilizing 1003 MB of GPU memory during inference. Logo detection accuracy maintains consistency across various scales, with detection rates of 91.2% for logos larger than 64×64 pixels and 86.5% for logos between 32×32 and 64×64 pixels. The complete system demonstrates robust performance across varying lighting conditions, achieving detection stability with illumination variations of ±2 EV stops while maintaining mAP above 85%.

Data Loading and Image Processing for YOLOv8n

The initial step in using the created datasets was to customize them and train several models to measure their accuracy. Because the primary motive was to detect the logo, the method was applied to create pictures of each company's logo from different points of view. Because the images with logos from the different companies had been taken from different devices, the JPG, JPEG, and PNG format images were chosen to simplify the process. On average, 30-35 pictures were taken for each company. Using Python environments, the images were converted to 200 with different angles and focus labels. Eight companies were initially chosen for the datasets, including 1063 images.

```
1. 1.  # Create an instance of ImageDataGenerator with augmentation options
2. datagen = ImageDataGenerator(
3.     rotation_range=40,          # Random rotation up to 40 degrees
4.     width_shift_range=0.2,      # Shift image horizontally by 20%
5.     height_shift_range=0.2,     # Shift image vertically by 20%
6.     shear_range=0.2,            # Shear angle
7.     zoom_range=0.2,             # Zoom in/out by 20%
8.     horizontal_flip=True,       # Randomly flip the image horizontally
9.     fill_mode='nearest'         # Fill mode for empty pixels after transformations
10. )
```

After that, all the pictures were annotated in the CVAT (Computer Vision Annotation Tools) platform, where each image was annotated individually to get a good outcome. A new folder with images and labels was created to train the model. To follow the sequence, image and labels maintain the serial to avoid negative detection.

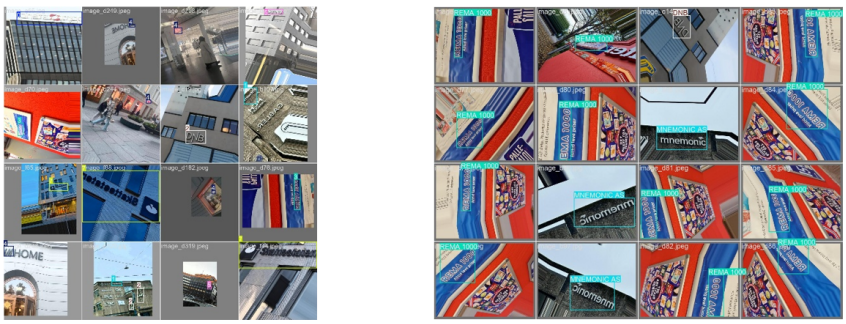


Figure 2.30: Annotation mark for Company logos.

Training, Testing, and Validation for Logo Detection

To train the model, the YAML file with the name config was created which was a script

that showed the path of the datasets created for the model. The file scripts the classes serially to avoid any disturbance during the training period.

```
1. 1.  # Classes
2. names:
3.   0: SOPRA STERIA AS
4.   1: MNEMONIC AS
5.   2: DNB
6.   3: REMA 1000
7.   4: H&M
8.   5: INTILITY AS
9.   6: VIPPS
10.  7: SKATTETATEN AS
```

Before initiating the deep learning for the YOLO model the required library need to be add. For model YOLO, 'ultralytics' need to be imported.

```
1. 1.  from ultralytics import YOLO
```

It is also essential to select exact variants for the training process that can be fit for the machine. For the process limitation in the machine, YOLOv8n was chosen for the final training.

```
1. 1.  # Load a model
2. model = YOLO("yolov8n.yaml")
```

A selected model indicated the images with their classes to start the training. The directory was created to save the trained model with different learning rates and epochs.

```
1. 1.  # Use the model
2. results = model.train(data="C:/Own
use/Login_Exercise/Datasets/Companies/New/Yolo_Data/config.yaml", epochs=40,
3.                       project="C:/Ownuse/Login_Exercise/Datasets/Companies/New/Yolo_Results",
# Directory to save results
                         name="custom_training_results_8Companies_lr0001(40)", lr0=0.0001
)  #train the model
```

To test the models, multiple images and videos were manually created, including logos from the datasets, to see if they could detect objects. To get real-time object detection, testing with a video that includes pixel and time sequences is important.

Performance & Optimization Strategy

Critical challenges addressed:

- Training data:
  - Limited dataset availability
  - Annotation complexity
  - Data augmentation needs and processes requiring multiple rounds of work.
- Model Performance:
  - Real-time detection requirements
  - Resource constraints
  - Accuracy vs speed trade-offs

2.3 Deployment

Based on the hardware constraints presented in the requirements during the initial planning stage of the project, the need for low power consumption requires a strategic approach. Running Large Language Models (LLMs) on limited resources, like a Raspberry Pi posts a unique challenge, and hence the planned approach is focused on balancing performance with energy efficiency ensuring smooth operations without overwhelming the hardware.

2.3.1 Dockerization of Components

In order to streamline the deployment and improve resource management, the core components of the application were containerized using Docker. The docker file being the main agent that acted as the source of deployment scripts for each of the components provided advantages including portability, isolation, and efficient resource utilization. Frontend, the ReactJs client was packaged into a lightweight Docker container, enabling quick and efficient deployment without impacting system performance. Similarly, the backend (Middleware), the Spring Boot application was also containerized, allowing it to run independently and handle requests efficiently. Docker's support for multi-threaded applications ensured smooth operations, even on limited hardware. Finally, both MySQL and MongoDB databases were containerized to maintain data persistence and scalability. Using Docker volumes, the databases were configured to optimize disk usage and minimize power consumption.

2.3.2 LLM Deployment Strategy

In the initial phase of the project, as the Pi-Farm was still being set up, the first course of action was hardware procurement i.e. buying a Raspberry Pi with the same specification compared to the ones in the Pi-Farm. Collectively as a group we bought a Raspberry Pi 5, with 16 GB of RAM and 128 GB of storage along with the cooling kit and casing. This helped us get hands on experience in getting started with running basic LLMs as the entire concept of running LLMs was new to the members of the cloud team.

Getting Stated with Ollama

Once we had access to the Raspberry Pi, the first thing to do was to run an LLM locally on the Pi, and the most popular way to do so, based on our research on the web, is through Ollama. Ollama is a command line interface tool using which one can easily download and run opensource LLMs privately unlike the readily available frontier models such as ChatGPT or Claude where the LLMs run in the cloud and hence you have no control over your private data. Moreover, it also streamlines the process to get any LLM up and running by taking care of the weights, the configuration files and the software dependencies. Ollama itself utilizes the popular open source llama.cpp library, which is optimized C++ code designed for faster inference and efficient memory management.

Setting up Ollama was very straightforward, installation via a simple shell script. Once it was installed the first model Tinyllama was pulled and then we ran inference on it. In the beginning we started with simple questions such as "What is the capital of Norway?", "How many planets are there in our solar system?" and so on. We would run

different models and do a simple qualitative analysis about the speed and coherence of the response. However, this strategy was not very useful as we did not have any metrics to compare one model to the other, more specifically the performance of one model to the other on the Raspberry Pi. That's why we opted for performance benchmarking with Ollama's built in run feature with the –verbose (Fig.2.31) flag that provides metrics such as total duration, load duration, prompt eval count, prompt eval duration, prompt eval rate, eval count, eval duration and eval rate.

```
apcha3338@raspberrypi:~ $ ollama run tinyllama:latest --verbose
>>> What is the capital of Norway?
The capital of Norway is Oslo, which is located in the northeastern part of the country.

total duration:       1.84085163s
load duration:        14.470751ms
prompt eval count:    41 token(s)
prompt eval duration: 108.642ms
prompt eval rate:     377.39 tokens/s
eval count:           23 token(s)
eval duration:        1.586157s
eval rate:            14.50 tokens/s
```

Figure 2.31: Sample benchmark of tinyllama with ollama

The most important metric to consider is the eval rate, which in this case is around 14.5 tokens per second. This tells us that the model can process a little over 3 words per second on average or around 180 words per minute. Compared to general conversational speech, which is around 120 – 200 words per minute, this is an excellent token generated per second speed running on a single Raspberry Pi device. A comparison table of the different LLMs tested over a single Raspberry Pi is given below:

Language Model	**Eval rate (tokens/second)**
Tinyllama (1.1b)	14.5
Gemma:2b	5.47
Phi3.5:3.8b	3.72
Llama3.2:1b	9.07
Qwen:0.5b	27.84

Table 2.1: Evaluation rate of different language models

Based on the above table, there is a direct correlation between the number of parameters and speed of generation of tokens (except for the llama3.2 family). All these benchmarks were performed with the same question; however, this alone cannot be the metric to evaluate an LLM as this does not tell us about other subjective parameters such as coherence (if the LLM is giving relevant output or gibberish), brevity (for a simple question, does the LLM give a simple answer or does it keep on generating more tokens; not relevant to the question). These parameters were part of the qualitative analysis and using the combined evaluation we found out that Llama3.2 and Qwen LLM family were the best compromise between quality of the response and the eval rate. Another thing to note here is that both are the latest offering from their creators which shows that newer generations of LLMs, even with same or lesser number of parameters are much faster as well as coherent than the previous generation. This shows that over time the training techniques for LLMs have been improving. We also found that the eval rate dropped significantly for

any model above 3 billion parameters and bigger models such as the llama3.1:8b dropped as low as 2 tokens per second. We did find some improvement by using 4-bt quantized models; however, a single Raspberry Pi was a good fit for an LLM less than or equal to 3 billion parameters.

Tensor Parallelism

Since a single Raspberry Pi could not support bigger models, we experimented with Tensor Parallelism i.e. distributed computer units performing calculations in parallel to allow bigger models such as the llama3.1:8b with higher token generation rate on small devices. Primarily we tried two different options for this:

- b4taz/distributed-llama [16] – An opensource repository designed to run LLMs on weak devices by distributing workload and memory usage. The project only supports the llama family of models and works with a setup of 2^n Nodes (for example, 4 nodes – 1 master and 3 worker), where the nodes synchronize with each other using TCP sockets. For a POC, we did performance benchmarking with 4 Raspberry Pis and ran the 8 billion parameter llama3.1 model as shown in Fig.2.32

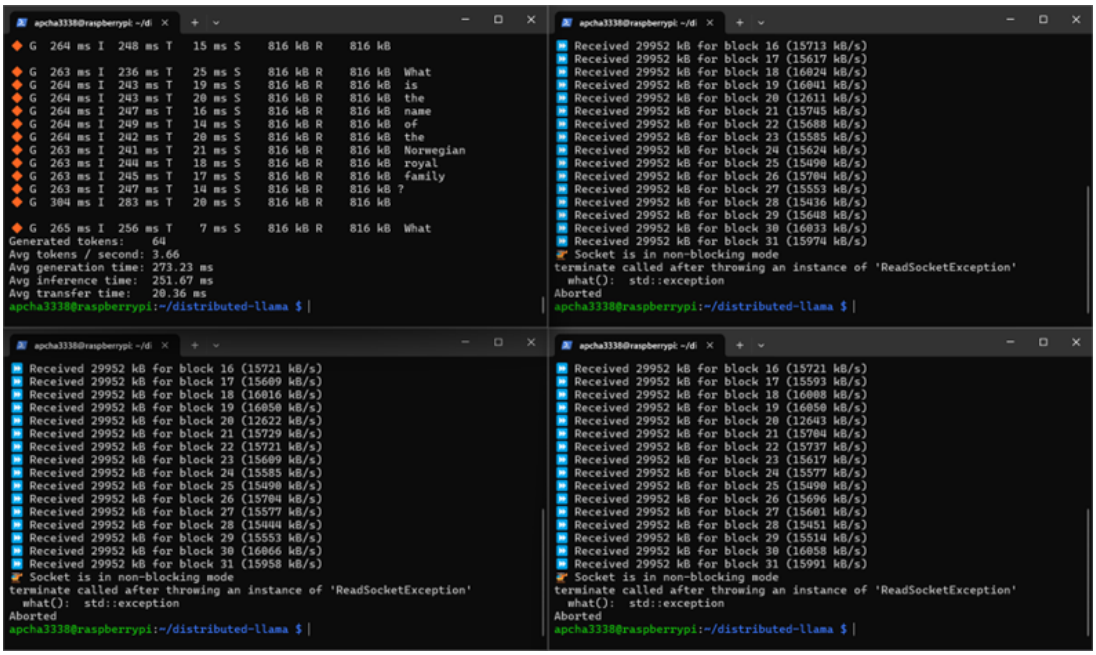


Figure 2.32: distributed-llama running with 1 master and 3 workers

The figure above shows the terminal view Pi40, 39, 38 and 37 respectively, where the top left node (Pi40) is the master node. We were able to achieve around 4 tokens per second on the distributed-llama setup which was twice than observed on a single Pi. However, the coherence of the model was much worse compared to the newer Llama3.2 models.

- exo-explore/exo [17] - Another opensource project that takes the concept of distributed computing on weak devices even further as the maintainers claim that you can even utilize your home devices such as smartphones, smartwatches, tablets, old laptops etc. to combine and form a virtual GPU which can be used to run LLMs

(even the 405 billion parameter llama model could be run in this way, in theory). We took a similar setup compared to distributed-llama with 4 Raspberry Pis shown in Fig.2.33

Figure 2.33: exo running on 4 Raspberry Pis

However, despite numerous attempts we were not able to run inference successfully due to multiple errors as shown in Fig.2.34 and the project itself is in very early stages with around 200 open issues on their GitHub project. By this time, we had started achieving some success with the smaller models such as the newer generation llama3.2:3b and thus to save time and effort we decided to cease any further research in to exo.

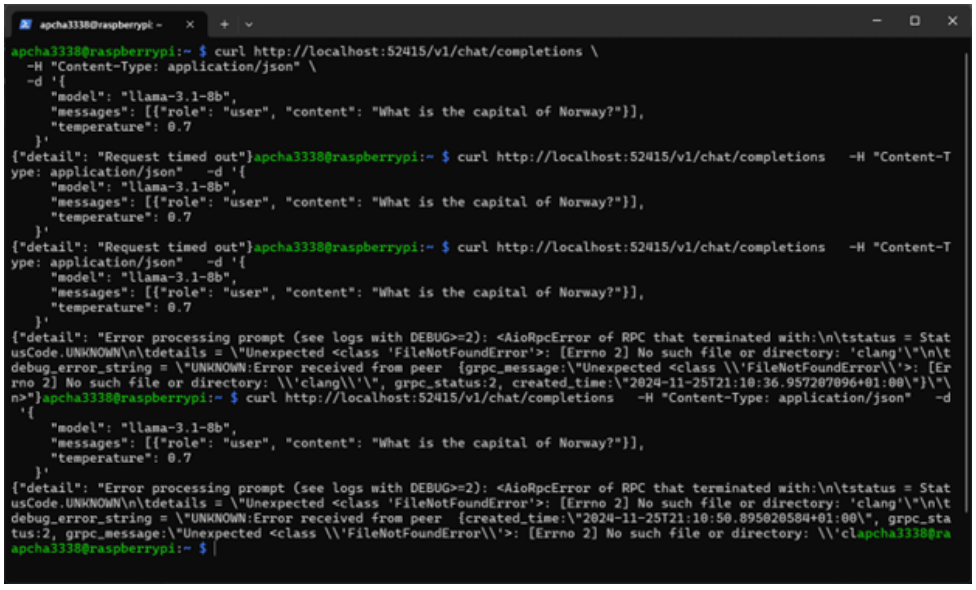


Figure 2.34: exo failed requests

3. Process Documentation

3.1 Project Management

The first part of the project was to agree upon creating a team structure or hierarchy withing the group of 17 students. Theoretically, it might be possible for one person to directly coordinate with more than a dozen people, but practically an ideal team size is somewhere between 5 to 9 members, based on the SCRUM methodology (more on that later). Since the group was composed of students from different specializations it felt natural to have partitions within the team based on course specialty such as the Cloud team composed of students from Cloud-based services and Operations, whose primary responsibility was to cover any deployment or software configuration activities. Similarly, we had the AI team from the Artificial Intelligence specialization and the Data Science team from the Data Science specialization; where each team would appoint one member as a representative/lead for that team. However, we wanted to give people the freedom to work on tasks from different specializations as this would allow them to cross skill themselves and share their competence with other teams. Therefore, we decided to go with a hybrid approach of SCRUM and the teams based on specializations. So, along with the core competency teams we also created the development team, the design team and the management team.

In the beginning we tried to follow the SCRUM methodology strictly by planning bi-weekly Sprints; conducting Sprint Review and Sprint Retrospective meetings followed by the Sprint Planning meeting every other Tuesday along with Daily Standup of 15 minutes every day to share the progress on individual tasks being worked upon. To conduct all these meetings and to keep track of the tasks we tried to experiment with popular collaboration and documentation tools such as JIRA, Notion, Trello etc. but we ended up settling for the Oslomet Microsoft Office 365 suite. The reasons for this decision are as follows:

- Ease of access and availability – Every user in the project already has an Office 365 account provided by the university. This saved time in creating new accounts on a different platform as well as saving costs since other platforms require paid subscriptions.
- Centralized communication and collaboration – With tools such as Teams, Outlook for conducting meetings and their integration with task management with Planner along with OneNote for documentation; everything that we needed was available on a single platform.
- Cloud storage and file sharing - The availability of cloud storage such as OneDrive allowed users to store and backup their data throughout the course of the project. The integration of OneDrive with other Office365 apps also allowed one to share files among different teams.

However, we quickly realized that this setup would not be suitable for our case as unlike a typical office environment students have other commitments such as other subjects or part time work, therefore we tried to offload the responsibility to conduct SCRUMs up to individual teams (sub teams such as Cloud would conduct their own bi-weekly SCRUM). But this did not work out as well because it created a lot of work for the team representatives and the feedback from most of the members was that a lot of time was being wasted on conducting the meetings rather than working. Finally, we decided to work on a trust-based model where we would only meet in person once a week at the designated time for the lecture, but we would maintain a Teams channel called "Daily Standup" where every member would address three main questions:

- What did I do yesterday?
- What am I doig today?
- Are there any blockers or impediments?

This helped us in keeping track of individual progress and since this channel was public, everyone could see what everyone else was working on, hence reducing the chances of two people working on the same task. Apart from the Daily Standup channel we also created different channels to reduce noise for all team members such as: General, Artificial Intelligence, Cloud and Development, Data Science, Oslomet Pi Farm and Project Discussion.

One of the biggest challenges that we had during the entire project was to define tasks because we needed to make something abstract into something that is measurable and achievable. For example, in the early stages of the project we had a lot of "research" tasks where say, a member of the AI team would research about a particular LLM. Without any predefined scope of the task, this could potentially go on for weeks or even months depending on how in depth the research is going to be. Therefore, we tried to adopt the SMART goals strategy i.e. every task must be specific, measurable, achievable, relevant and time bound. The task would either be defined by the team representative or by any individual keeping the SMART criteria in mind. Therefore, the above example of researching a particular LLM would become: "Research the capabilities, limitations and application of Gemma:2b with respect to personal career coach. Prepare a 10-slide presentation to demonstrate your research. Spend 1 hour every day until Monday next week, which is the deadline." The management team and the representative's primary responsibility here was to make sure that every task defined is recorded on the Planner and is followed up regularly. The duration of the task was left up to the individual themselves, but whatever they chose was enforced by the management team. A lot of emphasis was placed on the definition of done, where every task must have a finish line which is clearly defined so that it can be tracked to see if it is complete or not. In case someone is stuck or needs help with a task, they would be free to post a message on one of the multiple Teams channels.

Although we had agreed that we would adhere to the team structure discussed above along with task definitions, we did not have a written and signed contract that would enforce all the promises that we have made to each other. Therefore, we created a contract that covers the following: the project description, the team structure, the communication plan, conflict resolution, deadlines and time management, accountability, grading expectations. The terms of this contract were agreed upon and every group member signed

the contract henceforth adhering to the said terms. The contract can be found in the Appendix.

3.2 Timeline and Milestones

The project started in the second half of August and was due late November giving us roughly 12 weeks where we started from scratch and the target was to finish with a minimum viable product (MVP). After a slow start, where we had to agree upon an idea, learn the pre-requisite knowledge about new technologies, we were able to agree upon some distinct milestones, however, due to compressed timeline some planned features of the MVP could not be fully implemented. An overview of the timeline has been discussed below.

3.2.1 Project Initiation and Planning (Weeks 1- 3)

It took us about 3 weeks to agree upon an idea, creating a persona, establish the team structure, selecting the appropriate collaboration tools, setting up the work contract and prioritizing critical features; as discussed in the Project Management section. Although we had a slow start, due to factors beyond our control such as students dropping off the course, we could not proceed as fast as we wanted in the initial phase of the project. We also used this time to procure hardware (i.e. buy our own personal Raspberry Pi to test and work upon). However, by the end of the 3rd week we had a well-defined team structure and a clear problem statement to work upon with a fixed persona or in other words, what we are doing and who we are making it for.

3.2.2 Research and System Design (Weeks 4 - 6)

For the next three weeks we started to research about different LLMs which we could potentially run on our Raspberry Pi, learn about fine-tuning techniques, gathering datasets for different use cases such as training the resume analyzer expert, gather pictures of company logos to train the logo classifier, researching about diffusion models to generate and manipulate images of logos. In parallel we also conducted brainstorming sessions to create the user journey, frontend/ backed design and choose a technology stack for the same. This phase went longer than expected as we had multiple versions of UI designs, and it took us longer than expected to have a consensus, but, by the end of week 6 we had a well-defined user journey, a UI design proposal that satisfied the proposed user journey and we had gained some competence on fine tuning LLMs.

3.2.3 Prototype Development (Weeks 7-10)

The next four weeks were spent on different teams developing individual features of the application such as the cloud team preparing the underlying infrastructure to run the application and different models, the AI team fine-tuning the three experts, the development team creating the frontend based on the UI design proposals and the data science team developing the logo classifier. These were the core system components which took about a third of the total time available for the entire project. However, by the end of week 10, we had multiple LLMs running on the Raspberry Pi farm (distributed and per instance), 3 fine-tuned LLM experts, company logo classifier able to classify a limited

number of company logos, a functional but partially completed frontend capable of analyzing an uploaded resume and a job scraper program to get relevant job postings from the Arbeidsplassen API.

3.2.4 Finalization and Documentation (Weeks 11-12)

At this point we had developed the individual components of the application that needed to be integrated and packaged as one unified application. However, due to time constraints, assessment of other subjects and prioritization of the documentation for the final report, we were unable to fulfill our planned milestones in the final phase of the project. Although the project was marked by delays in the initial phase of the project which carried over, better planning and task estimation would have allowed us to complete the things that we missed out on.

4. Conclusions and Discussion

We started with this project aiming to create fully functioning minimum viable product within the given timeline, however, due to compressed timeline we were not able to reach our end goal in time. The team did not compromise on any of the features that were agreed upon, which led to delays in every aspect of the development process. Although we were able to develop different components of the application independently, it was not possible to integrate them together in time. The features that were successfully implemented are: we achieved some success with fine-tuning different agents, we were able to train an image classifier on a limited set of company logos, we were able to integrate the Resume Analyzer expert into the frontend and we were able deploy multiple instances of standalone and distributed models on several Raspberry Pis. Despite not making any significant contributions to AI-driven career coaching, we gained a lot of insights in the world of fine-tuning LLMs, running sLLMs on edge devices or weak computing devices, design and development of a large-scale project from scratch:

- Models under 3 billion parameters performed optimally on a single Raspberry pi
- Ditributed computing across multiple Raspberry Pi's doubled processing speed (4 tokens/second vs 2)
- Newer generation models(Llama3.2, Qwen) consistently outperformed older versions despite similiar parameter counts.
- RAG architecture proved essential for maintaining context while also maintaining resource constraint

For future work, features such as

- Generation of CV/Resume and cover letters based on user's data
- Job tracking for metrics for successfully getting a job or not(including ones for stages of the job application, progress on customized CV's and cover letters and preperation for a particular job description).
- Autonomous job application fulfillment on different platforms like Workday, Finn, LinkedIn etc.
- Schedule manager integrated with calendar applications to schedule interviews
- Social media manager for making relevant connections and networking can be implemented to enhance the user experience and expand on the vision of the vision detection system which would have enabled connections through just a logo on the street.

- A system for employers to filter out applicants (pre-screening would have been done for them) using the same feature set but in a different setting.

The application can be expanded to broader region i.e. focusing on a wider job market such as the entirety of Norway or even further to Europe, Asia or North America. With technological advancements in the field of AI, we get newer generations of LLMs that are much more efficient which makes it possible to fit a lot more agents onto a single device. Expanding the *modalities with audio* would also help improve the user experience, for example, editing your resume by simply talking. Potential *partnerships with existing platforms* such as LinkedIn would take this concept to the next level as an entire digital persona could be created based on your profile which could do all the tasks, mentioned above, for you with just a click of a button. An unexpected benefit of this approach would be offloading any GDPR concerns to LinkedIn or any other such platform, as the application would run on their platform. Although there are many positive aspects of this approach, it does lead one to think about a world where people never interact with each other digitally; only digital AI personas communicate with each other. Another thing to think about is instead of looking at the problem from John's perspective, it can be seen from the company's perspective and how can a company find the right "John" for them. Reflecting on what could have been done differently, having two independent groups would have been a better strategy than having a single group of 17 students. A lot of time was spent on getting everyone to agree on a single decision rather than having an absolute concrete vision and then executing it in a timely fashion. It would have also been beneficial to have some hands-on experience especially with fine tuning LLMs before we started to work on the project as there is a level of technical expertise that is required, and a lot of time was spent to acquire those skills during the project. With respect to project management, using strategies such as SMART goals, creating fixed agendas for meetings and traditional frameworks such as SCRUM really helped to achieve some order out of chaos. Instead of a hybrid trust-based model, we should have continued for the original SCRUM approach.

Bibliography

[1] Teal, "Ai resume builder," 2024, accessed: 2024-11-30. [Online]. Available: https://www.tealhq.com/

[2] Aragon, "The leading ai headshot generator for professionals," 2024, accessed: 2024-11-30. [Online]. Available: https://www.aragon.ai/

[3] Yoodli, "Land your dream job," 2024, accessed: 2024-11-30. [Online]. Available: https://yoodli.ai/use-cases/interview-preparation

[4] R. Singh and S. S. Gill, "Edge ai: a survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.

[5] A. Timilsina, "Yolov8 architecture explained," *Medium*, 2024. [Online]. Available: https://abintimilsina.medium.com/yolov8-architecture-explained-a5e90a560ce5

[6] ——, "YOLOv8 Architecture Explained! — abintimilsina.medium.com," https://abintimilsina.medium.com/yolov8-architecture-explained-a5e90a560ce5, [Accessed 30-11-2024].

[7] "What is YOLOv8? A Complete Guide. — blog.roboflow.com," https://blog.roboflow.com/what-is-yolov8/, [Accessed 30-11-2024].

[8] G. Boesch, "YOLOv8: A Complete Guide [2025 Update] - viso.ai — viso.ai," https://viso.ai/deep-learning/yolov8-guide/, [Accessed 30-11-2024].

[9] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications," *arXiv preprint arXiv:2410.11845*, 2024.

[10] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[12] Z. Zhang, X. Han, Z. Liu, X. Jiang, M. Sun, and Q. Liu, "Ernie: Enhanced language representation with informative entities," 2019. [Online]. Available: https://arxiv.org/abs/1905.07129

[13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[14] V. Sanh, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[15] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," 2023. [Online]. Available: https://arxiv.org/abs/2309.16609

[16] b4taz, "Distributed llama," 2024, accessed: 2024-11-30. [Online]. Available: https://github.com/b4rtaz/distributed-llama

[17] exo explore, "Run your own ai cluster at home with everyday devices," 2024, accessed: 2024-11-30. [Online]. Available: https://github.com/exo-explore/exo

A. Work Contract

ACIT4040 Project Work Contract

Low power, multimodal, mixture of experts Career Coach application

Project Description

The project's goal is to create a low-power LLM support system that can be deployed on Raspberry Pis. It must be multi-modal (vision capabilities), must have a local and a cloud component and must be mixture of experts/agents (at least 3 different fine-tuned agents). The application running on top of this system is a Career Coach application designed to help computer science graduates to find and prepare for entry level IT jobs in Oslo.

Team Structure

The primary team structure is based on individual specialties, with representatives for each team responsible for distributing tasks among their team members. However, individuals can/will participate in weekly scrum teams to help with other tasks which may change weekly. All members agree to complete their individual tasks on time, communicate regularly, and assist each other when needed.

Team	Representatives
Data Science	Alexandros Messaritakis Chousein Aga
AI	Mehdi Heidari & Rolf Alexander Klæboe
Cloud	Sacir Turkovic
Development	Yuvraj Singh

Other than the primary representatives, individuals can/will be assigned specific responsibilities throughout the project and must adhere to the same standards.

Communication Plan

- Primary Communication Tool – Teams, Email, Planner
- Response Time Expectation – Within 24 hours
- Weekly meeting time (all members) - Wednesday after class
- Daily Standup Meetings – All team members to post updates everyday answering three main questions on the “Daily Standup” Teams Channel
    - What am I working on today?

    - What did I work on yesterday?
    - Are there any blockers/impediments?
- Meeting Platform – Teams / In-Person

Conflict Resolution

In the event of a disagreement, group members agree to:

1. Discuss the issue among all members and POC with primary ownership
2. If unresolved, vote on the best course of action (majority wins)
3. If the issue remains unresolved, consult with instructor

Deadlines and Time Management

Each group member agrees to complete their tasks by the deadlines set. If a member anticipates not being able to meet a deadline, they will:

1. Notify the group at least 48 hours (about 2 days) in advance.
2. All members agree to update their own tasks regularly on Planner
3. Arrange for assistance from another group member, if needed.
4. Propose a plan to make up for the missed deadline.

Accountability

Each member agrees to contribute equally to the project. If a member consistently fails to contribute:

1. The group will first address the issue with the member directly.
2. If no improvement is made, the group will escalate the issue to the instructor.

Grading Expectations

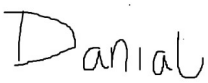
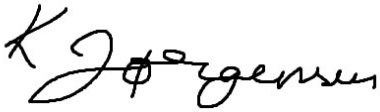
All members agree to:

- Dedicate 5 hours per week as per their availability
- Strive for a high standard of quality in the work produced.
- Share equal credit for the work submitted.
- Everyone aiming for the same grade i.e. **A**

Signatures

By signing below, all members agree to the terms outlined in this contract and commit to collaborating in a respectful and productive manner throughout the duration of the project

Name	Signature	Date
Apoorv Chaudhary		23/09/2024
Sacir Turkovic		24.09.202
Dung Thuy Vu		25.09.2024
Yuvraj Singh		25/09/2024
Lars Edvardsen		25/09/2024
Md Mahmudul Hasan		25/09/2024
Alexandros Messaritakis		25/09/2024
Mehdi Heidari		25/09/24

Kazi Umme Salma Ami		25/09/24
Fredrik Skatvedt		25/09/2024
Rolf Alexander Klæboe		25/09/2024
Ranya Mohamed Samy		25/09/24
Fredrik Andersen Langsem		25/09/24
Danial Khan		25.09.24
Ahmed Osman		25.09.24
Guneshwar Singh Manhas		30.09.2024
Kristian Jørgensen		29.11.24